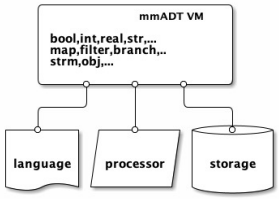
DT: The Virtual Machine

uez [creator] (RReduX) – Ryan Wisnesky [advisor] (Conexus)

mm-ADT is a dual licensed AGPL3/commercial open source project that offers software engineers, computer scientists, mathematicians, and others in the software industry a royalty-based OSS model. The Turing Complete mm-ADT virtual machine (VM) integrates disparate data technologies via algebraic composition, yielding *synthetic data systems* that have the requisite computational power and expressivity for the problems they were designed to solve. As an economic model, each integration point offers the respective development team access to the revenue streams generated by any for-profit organization leveraging mm-ADT.

Machine Components

M integrates the following data processing technologies:



g Languages: Language designers can create custom develop parsers for existing languages that compile to assembly code (`mmlang`) or bytecode (binary encoding

ngines: Processor developers can enable their push- or ecution engines to be programmed by any mm-ADT e abstract processing model supports single-machine, ed, agent-based, distributed near-time, and/or cluster-ch-analytic processors.

ems: Storage engineers can expose their systems via expressed in mm-ADT's dependent type system that enable the lossless encoding of key/value store, ore, wide-column store, graph store, relational store, and other novel or hybrid structures such as docu-graph, and quantum data structures.

M enables the intermingling of any language, any processor, and any storage system that can faithfully ore language semantics (*types and values*), processor semantics (*instruction set architecture*), and/or cs (*data structure streams*).

heory

nction

program denotes a single unary function that maps an `obj` of type S (start) to an `obj` of type E (end) n signature

$$f : S \to E.$$

s of mm-ADT are realized in the definition of an `obj` (which includes both types and values) and the re of an f-program (which is a composition of nested curried functions). The sole purpose of this is to make salient the various algebraic structures that are operationalized to ultimately yield the mapping

ebras

operations: addition ($+$) and multiplication ($*$). Abstract algebra is the study of these two operations across f slightly different generalized structures devoid of considerations regarding the particulars of their n: integers, digital circuits, quantum systems in a superposition, etc. Theoreticians deduce theorems based on the structures and experimentalists apply the structural patterns to other systems, real or designed.

al masterpiece of the discipline is the **magma hiearchy**.

, $*$): a set A with a potentially associative binary operator $* : A \times A \to A$.

$(A, *)$: a magma with an associative binary operator $(a * b) * c = a * (b * c)$.

, $*$, $\mathbf{1}$): a semigroup with an identity element such that $a * \mathbf{1} = a = \mathbf{1} * a$.

$*$, $\mathbf{1}$): a monoid with ever element having an inverse such that $a * a^{-1} = \mathbf{1} = a^{-1} * a$.

$+$, $*$, $\mathbf{0}$, $\mathbf{1}$): a $+$ -group and a $*$ -monoid bound by distributivity, $a * (b + c) = (a * b) + (a * c)$.

$+$, $*$, $\mathbf{0}$, $\mathbf{1}$): a ring where the $*$ -monoid is a $*$ -group.

ichments to these structures blur the sharp lines that divide them.

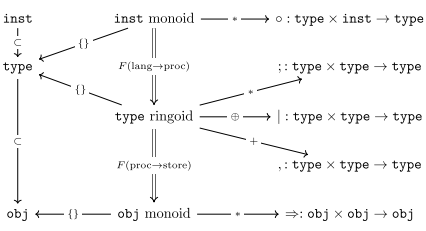
$b = b \cdot a$: the binary operator is agnostic to the order of the arguments.

t $a \cdot a = a$: the binary operator reaches a fixpoint on self compositions.

e binary operator is replaced with *concatenation* to record operation sequences, not evaluate them.

- Module $A[x]$: a binary magma that respects the abstract form of linear algebra via *scalars* and *vectors*.
- Polynomial $x_1 a + x_2 a^2 + ... + x_n a^n$: a binary magma with free addition and non-free multiplication.
- ...

No one practitioner will ever have a complete grasp of the intricacies that bind $+$ and $*$ together over A.



The base algebra of mm-ADT is a type-oriented ring algebra called the **obj stream ring**. There are three surjective homomorphisms from the `obj` stream ring to the algebras of the aforementioned components. The **language algebra**'s free monoid enables the nested, serial composition of parameterized instructions (`inst`) from the mm-ADT instruction set architecture and is called the `inst` monoid. The **processor algebra** is called the type ringoid and it is a free polynomial ringoid (`poly`) at compilation and non-free ringoid at evaluation. The **storage algebra** is called the `obj` monoid and it maintains a carrier set composed of all mm-ADT objects (`obj`) and an associative, binary operator for constructing data streams.

These component algebras represent the particular perspective that each component has on a shared data structure

called the **obj graph**. This graph has a faithful encoding as a generalized Cayley graph and a commutative diagram. It serves as the medium by which the virtual machine's computations take place: from specification, to compilation and then evaluation.

Component	Algebra
language	`inst` monoid
processor	`type` ringoid
storage	`obj` monoid

The primary purpose of this documentation is to explain these algebras, specify their relationship to one another and demonstrate how they are manipulated by mm-ADT technologies. Data system engineers will learn how to integrate their technology so end users may compose their efforts with others' to create *synthetic data systems* tailored to a problem's particular computational requirements.

mm-ADT Technology

mm-ADT Console

The mm-ADT VM provides a REPL console for users to evaluate mm-ADT programs written in any mm-ADT language. The reference language distributed with the VM is called `mmlang`. `mmlang` is a low-level, functional language that is in near 1-to-1 correspondence with the underlying VM architecture — offering it's users Turing-Complete expressivity when writing programs and an interactive teaching tool for studying the mm-ADT VM.

```
~/mm-adt bin/mmadt.sh
                                  _____ _______
                          /\     |  __ |__   __|
 _ __ ___  _ __ ___ _____ /  \ | |  | | | |
| '_ ` _ \| '_ ` _ |_____/ /\ \| |  | | | |
| | | | | | | | | |  / ____ \ |__| | | |
|_| |_| |_|_| |_| |_| /_/    \_\____/  |_|
                                  mm-adt.org
mmlang>
```

A simple console session is presented below, where the parser expects programs written in the language specified left of the `>` prompt. All the examples contained herein are presented using `mmlang`.

```
mmlang> 1
==>1
mmlang> 1+2
==>3
mmlang> 1[plus,2]
==>3
```

mmlang Syntax and Semantics

The context-free grammar for `mmlang` is presented below. Every `mmlang` expression denotes an element of the free `inst` monoid.

```
obj   ::= (type | value)q
value ::= vbool | vint | vreal | vstr
vbool ::= 'true' | 'false'
vint  ::= [1-9][0-9]*
vreal ::= [0-9]+'.'[0-9]*
vstr  ::= "'" [a-zA-Z]* "'"
type  ::= ctype | dtype
ctype ::= 'bool' | 'int'  | 'real' | 'str' | poly | '_'
poly  ::= lst | rec | inst
q     ::= '{' int (',' int)? '}'
dtype ::= ctype q? ('<=' ctype q?)? inst*
sep   ::= ';' | ',' | '|'
lst   ::= '(' obj (sep obj)* ')' q?
rec   ::= '(' obj '->' obj (sep obj '->' obj)* ')' q?
inst  ::= '[' op(','obj)* ']' q?
op    ::= 'a' | 'add' | 'and' | 'as' | 'combine' | 'count' | 'eq' | 'error' |
          'explain' | 'fold' | 'from' | 'get' | 'given' | 'groupCount' | 'gt' |
          'gte' | 'head' | 'id' | 'is' | 'last' | 'lt' | 'lte' | 'map' | 'merge' |
          'mult' | 'neg' | 'noop' | 'one' | 'or' | 'path' | 'plus' | 'pow' | 'put' |
          'q' | 'repeat' |'split' | 'start' | 'tail' | 'to' | 'trace' | 'type' | 'zero'
```

The Type

Products and Coproducts

Category theory is the study of structure via morphisms that expose (or generate) other structures. Two important category theoretic concepts dealing with substructures are **products** and **coproducts**.

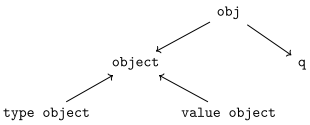
A product is any object defined in terms of it's accessible component objects. That is, from a single object, via π_n projection morphisms, the product is decomposed into it's constituent parts.

A coproduct is any object defined in terms of the component objects used to construct it. That is, from many objects, via ι_n injection morphisms, a coproduct can be composed from constituent parts.

Along with these decomposition (and composition) morphisms, there exists an isomorphism between any two products (or coproducts) should they project (or inject) to the same component objects. That is, product and coproduct equality are defined via component equality.

Types and Values

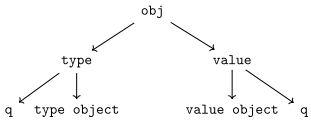
Everything that can be denoted in `mmlang` is an `obj`. Within the VM and outside the referential purview of an interfacing language, every `obj` is the product of



1. an **object** that is either a **type object** or a **value object** and
2. a **quantifier** specifying the "amount" of objects being denoted.

$$\begin{aligned} \texttt{obj} &= \texttt{object} &&\times \texttt{q} \\ \texttt{obj} &= (\texttt{type object} + \texttt{value object}) &&\times \texttt{q}. \end{aligned}$$

This internal structure is well-defined as an algebraic ring. The ring axioms specify how the internals of an `obj` are related via two binary operators: $\times$ and $+$. One particular axiom states that products both left and right distribute over coproducts. Thus, the previous formula is equivalent to



$$\texttt{obj} = (\texttt{type object} \times \texttt{q}) + (\texttt{value object} \times \texttt{q}).$$

There are two distinct kinds of mm-ADT `objs`: *quantified type objects* and *quantified value objects*. These products of the `obj` coproduct are called by simpler names: **type** and **value**. That is the **obj meta-model**.

$$\texttt{obj} = \texttt{type} + \texttt{value}$$

There are only a few instances in which it is necessary to consider the object component of an `obj` separate from its quantifier component. The terms *type* and *value* will always refer to the object/quantifier-pair as a whole — i.e., an `obj`.

```
mmlang> int                 ❶
==>int
mmlang> 1                   ❷
==>1
mmlang> int{5}              ❸
==>int{5}
mmlang> 1{5}                ❹
==>1{5}
mmlang> ['a','b','a']       ❺
==>'b'
==>'a'{2}
```

❶ A single `int` type.

❷ A single `int` value of `1`.

❸ Five `int` types.

❹ Five `1` `int` values.

❺ A `str` **stream** composed of `'a'`, `'b'`, and `'a'` (definition forthcoming).

Both types and values can be operated on by types, where each is predominately the focus of either compilation (types) or evaluation (values).

- $(\texttt{type} \times \texttt{type}) \rightarrow \texttt{type}$: Used in compilation for type inferencing and type rewriting, and
- $(\texttt{value} \times \texttt{type}) \rightarrow \texttt{value}$: Used in program evaluation and as lambda functions.

```
mmlang> int => int[is,[gt,0]]                  ❶
==>int{?}<=int[is,bool<=int[gt,0]]
mmlang> 5 => int{?}<=int[is,bool<=int[gt,0]]   ❷
==>5
```

❶ **Compilation**: The `int`-type is applied to the `int[is,[gt,0]]`-type to yield a *maybe* `int{?}`-type.

❷ **Evaluation**: The nested `bool<=int[gt,0]`-type is a lamba function yielding `true` or `false`.

Some interesting conceptual blurs arise from the intermixing of types and values. The particulars of the ideas in the table below will be discussed over the course of the documentation.

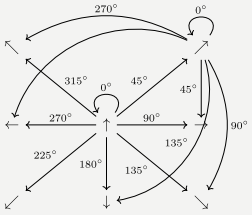
Table 1. Consequences of Type/Value Integration

structure A	structure B	unification
type	program	a program is a "complicated" type.
compilation	evaluation	compilations are type evaluations, where a compilation error is a "type runtime" error.
type	variable	types refer to values across contexts and variables refer to values within a context.
type	AST	a single intermediate representation is used in compilation, optimization, and evaluation.
type	function	functions are (dependent) types with values generated at evaluation.
state	trace	types and values both encode state information in their process traces.
classical	quantum	quantum computing is classical computing with a unitary matrix quantifier ring.

Type Structure

Cayley Graphs

A Cayley graph is a graphical encoding of a group. If $(A, \cdot, I)$ is a group with carrier set A, binary operator $\cdot : (A \times A) \rightarrow A$, and generating set $I \subseteq A$ then the graph $G = (V, E)$ with vertices $V = A$ and labeled edges $E = A \times I \times A$ is the Cayley graph of the group. The directed edge $(a, i, b) \in E$ written $a \rightarrow_i b$ states that the vertices $a, b \in A$ are connected by an edge labeled with the element $i \in I$. Thus, $a \rightarrow_i b$ captures the group operation $a \cdot i = b$.



When constructed in full, a Cayley graph's vertices are the group elements and its edges represent the set of all possible I-transitions between elements. When lazily constructed, a Cayley graph encodes the history of a group computation, where the current element has an incoming I-edge from the previous element. A Cayley graph captures both the proto-free and non-free aspects of a group. The non-free aspect is realized by any edge $e = (a, i, b)$ such that $ai \mapsto b$ and an element of the corresponding free algebra $(A^*, *)$ can be constructed by concatenating the edge labels of a path $\prod_{e \in (a,i,b)^*} \pi_1(e)$.

A *generalized* Cayley graph does not require that every $i \in I$ have a corresponding $i^{-1} \in I$ such that $i \cdot i^{-1} = \mathbf{1}$ (i.e., multiplicative inverses). By lifting this constraint, the Cayley graphical structure can be used to encode other magmas such as monoids and semigroups.

An `obj` is either a type or a value:

$$\texttt{obj} = \texttt{type} + \texttt{value}.$$

That equation is not an axiom, but a theorem. Its truth can be deduced from the equations of the full axiomatization of `obj`. In particular, for types, they are defined relative to other types. Types are a coproduct of either a

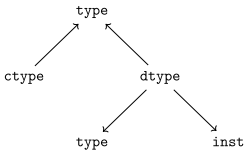
1. **canonical type** (ctype): a base/fundamental type, or a
2. **derived type** (dtype): a product of a type and an instruction (`inst`).

The ctypes are nominal types. There are five ctypes:

1. **bool**: denotes the set of booleans — $\mathbb{B}$.
2. **int**: denotes the set of integers — $\mathbb{Z}$.
3. **real**: denotes the set of reals — $\mathbb{R}$.
4. **str**: denotes the set of character strings — Σ^*.
5. **poly**: denotes the set of free objects — $\texttt{obj}^*$.

The dtypes are structural types whose recursive definition's base case is a ctype realized via a chain of instructions (`inst`) that operate on types to yield types. In other words, instructions are the generating set of a type monoid. Formally, the type coproduct is defined as

$$\begin{aligned}\texttt{type} &= (\texttt{bool} + \texttt{int} + \texttt{real} + \texttt{str} + \texttt{poly}) + (\texttt{type} \times \texttt{inst}) \\ \texttt{type} &= \texttt{ctype} + (\texttt{type} \times \texttt{inst}) \\ \texttt{type} &= \texttt{ctype} + \texttt{dtype}.\end{aligned}$$

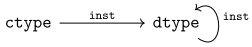


Every `obj` has an associated quantifier. When the typographical representation of an `obj` lacks an associated quantifier, the quantifier is unity. For instance, the `real 1.35{1}` is written more economically as `1.35`.

A dtype has two product projections. The **type projection** denotes the domain and the **instruction projection** denotes the function, where the type product as a whole, relative to the aforementioned component projections, is the range.

$$\begin{gathered}\texttt{type} = (\texttt{type} \quad \times \quad \texttt{inst}) \quad + \texttt{ctype} \\ \text{“range} = (\text{domain and function}) \text{ or } \text{base”}\end{gathered}$$

The implication of the dtype product is that mm-ADT types are generated inductively by applying instructions from the mm-ADT VM's instruction set architecture (`inst`). The application of an `inst` to a type (ctype or dtype) yields a dtype that is a structural expansion of the previous type.



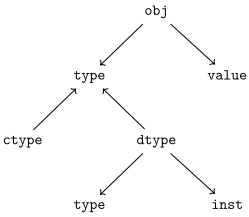
For example, `int` is a ctype. When `int` is applied to the instruction `[is>0]`, the dtype `int{?}<=int[is>0]` is formed, where `[is>0]` is syntactic sugar for `[is,[gt,0]]`. This dtype is a refinement type that restricts `int` to only those `int` values greater than zero — i.e., a natural number $\mathbb{N}^+$. In terms of the "*range = domain and function*" reading, when an `int` (**domain**) is applied to `[is>0]` (**function**), the result is either an `int` greater than zero or no `int` at all `{?}` (**range**).

$$\texttt{int} \xrightarrow{\texttt{[is>0]}} \texttt{int}\{?\}$$

The diagram above captures a fundamental structure in mm-ADT called the **obj graph**. The `obj` graph is used for, amongst other things, type checking, type inference, compiler optimization, and garbage collection. The subgraph concerned with type definitions is called the **type graph**. The subgraph encoding values and their relations as a function of the types is called the **value graph**. The `obj` graph is also the codomain of an embedding whose domain is an `obj` ringoid called the **stream ring**. Both the `obj` graph and stream ring form the primary topics of study in this documentation.

The `obj` meta-model structure thus far is diagrammed on the right (with quantifiers attached to each component). On the left are some example `mmlang` expressions.

```
mmlang> int                                                    ❶
==>int
mmlang> int{2}                                                 ❷
==>int{2}
mmlang> int{2}[is>0]                                           ❸
==>int{0,2}<=int{2}[is,bool{2}<=int{2}[gt,0]]
mmlang> int{2}[is>0][plus,[neg]]                               ❹
==>int{0,2}<=int{2}[is,bool{2}<=int{2}[gt,0]][plus,int{0,2}[neg]]
mmlang> 5{2} => int{2}[is>0][plus,[neg]]                       ❺
==>0{2}
```



❶ A ctype denoting a single integer.

❷ A ctype denoting two integers.

❸ A dtype denoting zero, one, or two integers greater than 0.

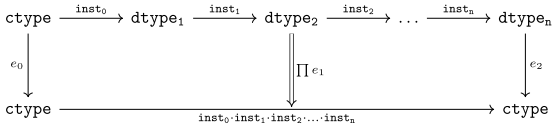
❹ A dtype extending the previous type with negative integer addition.

❺ A value of two fives applied to the previous type with the result being two 0s.

Type Components

The illustration below highlights the two primary components of a type, where an edge of the Cayley graph is the triple $e = (a, i, b) \in (\texttt{type} \times \texttt{inst} \times \texttt{type})$.

1. **Type signature**: the ctype specification of a type's domain and range.
2. **Type definition**: a domain rooted instruction sequence terminating at the range.



An image referred to as a **diagram** or commuting diagram is isomorphic to the system of equations it captures and thus, respects the axioms of the algebraic structure being diagrammed. An image referred to as an **illustration** is intended to elicit a realization of the associated topic via intuition and should not be considered a faithful encoding of an underlying mathematics.

Type Signature

Every mm-ADT type can be generally understood as a function that maps an `obj` of one type to an `obj` of another type. A **type signature** specifies the source and target of this mapping, where the **domain** is the source type, and the **range** is the target type. In `mmlang` a type signature has the following general form where `{q}` is the ctype's associated quantifier.

```
range{q}<=domain{q}
```

In common mathematical vernacular, if the function f has a domain of X and a range (codomain) of Y, then its signature is denoted $f\colon X \to Y$. Furthermore, with quantifiers in Q, the function signature would be denoted $f\colon X \times Q \to Y \times Q$ or $f\colon (X \times Q) \to (Y \times Q)$.

mmlang Expression	Description
`mmlang> int<=int` `==>int`	From the perspective of "*type-as-function*," An mm-ADT `int` is a no-op on the set of integers. Given any integer, `int` returns that integer. In `mmlang`, when the domain and range are the same, the `<=` and repeated type are not displayed. That is `int<=int` is more concisely displayed as `int`.

`mmlang> int{1}`
`==>int`
`mmlang> int`
`==>int`
	In most programming languages, a value can be typed `int` as in
	
	`val x:int = 10.`
	
	Such declarations state that the value referred to by `x` is a *single* element within the set of integers. The concept of a "single element" is captured in mm-ADT by the `obj` quantifier, where a unit quantifier is not displayed in `mmlang`. That is, `int{1}` is more concisely displayed as `int`.

`mmlang> int{5}` `==>int{5}`	`int{5}` is a type referring to 5 integers. As a point of comparison, `int{1}` refers to a single integer with a syntax sugar of `int` in `mmlang`.

`mmlang> int{0,5}`
`==>int{0,5}`
`mmlang> int{0,5}<=int{0,5}`
`==>int{0,5}`
	Quantifiers must be elements from a ring with unity. In the previous examples, the quantifier ring was $(\mathbb{Z},\ +\ ,\ *\ )$. In this example, the quantifier ring is $(\mathbb{Z} \times \mathbb{Z},\ +\ ,\ *\ )$, where the carrier set is the set of all pairs of integers and addition and multiplication operate pairwise,
	
	$$(a, b) * (c, d) \mapsto (a * c, b * d).$$
	
	The type `int{0,5}` denotes the inclusive range of 0, 1, 2, 3, 4, or 5 integers. In practice, the $(\mathbb{Z} \times \mathbb{Z})$ quantifier ring represents uncertainty as to the number of elements being referred to.

`mmlang> int<=bool` `==>int<=bool`	Types that are fully specified by their type signature are canonical types (ctypes). Therefore, `bool<=int` is meaningless as there are no instructions to map an `int` to a `bool`. This example is in the `mm` model-ADT, where given another model, it is possible for `bool<=int` to yield a result.

Type Definition

Commuting Diagrams

Category theory is a branch of abstract algebra that studies, among other things, arbitrary algebraic structures via their homomorphic embedding in a multi-sorted monoid called a **category**. A category $\mathscr{C}$ is denoted

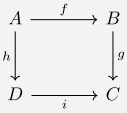
$$(\mathbf{C}, \circ, \mathbf{1}),$$

where $\mathbf{C}$ is a set-family of *morphisms*, $\circ : \mathbf{C} \times \mathbf{C} \to \mathbf{C}$ is an associative binary morphism *composition* operator, and for every *identity* morphism $\mathbf{1}_A \in \mathbf{1}, \mathbf{1}_A \circ \mathbf{1}_A = \mathbf{1}_A$ denotes an *object* that is more simply written A such that $A \mapsto \mathbf{1}_A$. The family set $\mathbf{C}$ indexes hom-sets with $\mathbf{C}(A, B)$ denoting all morphism between objects A and B, where $f\colon A \to B \in \mathbf{C}(A, B)$ and $A \to A \cong \mathbf{1}_A \cong A$.

Unlike classical monoids, a category's $\circ$ operator is generally not closed. That is, there are compositions which may not be defined. It is this aspect of a category that makes it a *multi-sorted* (or typed) monoid.

The discipline of category theory makes extensive use of a homomorphism from a category to a directed labeled graph called a diagram. These diagrams realize the same underlying unitary operation of the generators of a magma within a generalized Cayley graph. If $f\colon A \to B$ and $g\colon B \to C$, then there exists the morphism path

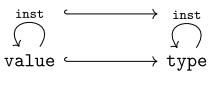
$$A \overset{f}{\to} B \overset{g}{\to} C,$$



which, in Cayley graph notation, is denoted $A \to_f B \to_g C$. An important subset of diagrams are the commutative diagrams. In a commutative diagram every morphism path starting at the same source and ending at the same destination are considered *equivalent* (with respects to equivalence in the respective algebraic structure being modeled categorically). Thus, if $g \circ f = i \circ h$, then it is said that the above diagram *commutes*.

Types and values both have a **ground** that exists outside of the mm-ADT virtual machine within the hosting environment (e.g. the JVM). The ground of the mm-ADT value `2` is the JVM primitive `2L` (a Java `long`). The ground of the mm-ADT type `int` is the JVM class `java.lang.Long`. When the instruction `[plus,4]` is applied to the mm-ADT `int` value `2`, a new mm-ADT `int` value is created whose ground is the JVM value `6L`. When `[plus,4]` is applied to the mm-ADT `int` type, a new type is created with the same `java.lang.Long` ground. Thus, the information that distinguishes `int` from `int[plus,4]` is in the reference to the instruction that was applied to `int`.

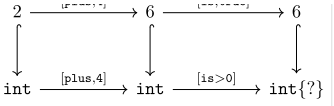
For a type, the deterministic chain of references is called the **type definition** and is encoded as a path in the **type graph**. For a value, the **value graph** encodes a path called the **value history**. The commutative diagram below is composed of two horizontal paths. The top path is a value history and the bottom path is a type definition. These paths are joined by the `[type]` instruction which are diagrammed using hook-tailed arrows that denote, by convention, a monomorphic embedding known more simply as an inclusion (i.e., $a \in A$ or $A' \subset A$). The set of all `[type]` morphisms is equivalent to the hom-set `Hom(value,type)` which defines a functor that specifies a particular embedding of the value graph into the type graph. This aggregate structure is of import in mm-ADT. It's called the **obj graph**.



In theory, the complete history of an mm-ADT program (from compilation to execution) is stored in the `obj` graph. However, in practice, the mm-ADT VM removes paths once they are no longer required by the program. This process is called **path retraction** and is the mm-ADT equivalent of garbage collection.

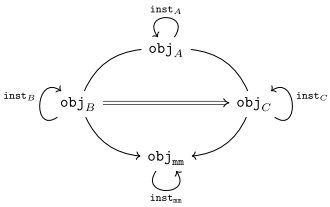
`mmlang> 2[plus,4][is>0][path]`	[plus,4] [is,true]

```
==>(2;[plus,4];6;[is,true];6)<=2[plus,4][is,true][path]
mmlang> int[plus,4][is>0][path]
==>(int;[plus,4];int;[is,bool<=int[gt,0]];int{?}){?}
<=int[plus,4][is,bool<=int[gt,0]][path]
mmlang> 2[plus,4][is>0][type]
==>int{?}<=int[plus,4][is,bool<=int[gt,0]]
```



In practice, the string representation of a value is its *ground* and the string representation of a type is its *path*.

To provide a preview of what is to come, an mm-ADT **model** defines an `obj` graph for a particular domain of discourse. A transition from model A to model B may be possible by way of a functor derived from $\texttt{Hom}(A, B)$. Furthermore, it may be possible to go from A to $\texttt{mm}$ via a composition with $\texttt{Hom}(B, \texttt{mm})$. Two such parallel compositions between models are illustrated in the associated diagram and written as



$$\texttt{Hom}(A, B) \circ \texttt{Hom}(B, \texttt{mm})$$
$$\texttt{Hom}(A, C) \circ \texttt{Hom}(C, \texttt{mm}).$$

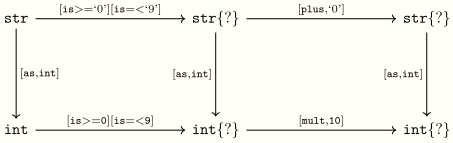
Model mappings allow types written in one universe to be evaluated within another universe, where, ultimately, all types must be grounded in the base `mm` model. The specification and selection of paths to `mm` is determined by mm-ADT programs that leverage model libraries. Ultimately, it is through `mm` that the mm-ADT VM communicates with storage systems and processing engines, enabling arbitrary models atop a sound evaluation.

Deep Dive 1. The Obj Graph as a Cayley Graph and a Commutative Diagram

The `obj` graph is both a generalized Cayley graph of a partial monoid and the commutative diagram (or quiver) of the category composed of `obj` vertices and `inst` labeled edges. More generally, the `obj` graph is the graph of unary functions comprising `inst`, where instructions operate on both types and values. From compilation to evaluation, depending on the particular context, either interpretation will be leveraged.

- **Commutative diagram**: vertices denote type/value-objects of the `obj` category with `inst` morphisms.

The `obj` graph's commuting property eases compile-time and runtime type rewriting. If two paths have the same source vertex (domain) and target vertex (range), then both paths yield the same result (the target vertex). In practice, evaluating the instructions along the computationally cheaper path is prudent.



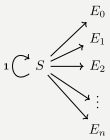
- **Cayley graph**: vertices denote type/value-elements of the `inst` monoid with generating edges in `inst`.

As a generalized, multi-rooted monoidal Cayley graph, the set of all possible mm-ADT computations is theoretically predetermined given the monoid presentation containing the root `objs` (e.g. the ctypes), its generators (`inst`), and relations (path equations). This static immutable structure serves to memoize computational results. This is especially useful when considering **streams** (definition forthcoming) and their role in data-intensive, cluster-oriented environments where storage is cheap and processors are costly.

Type Quantification

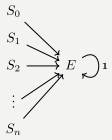
Initial and Terminal Objects

A category may have an initial and/or terminal object.



An **initial object** S is the domain of a set of morphism $S \to E_n$. Initial objects, via their morphisms, generate all the objects of the category. If there is an initial object, then it is unique in that if there is another initial object, it has the same diagrammatic topology — all outgoing morphisms and no incoming morphisms save the identity. Thus, besides labels, two initials are isomorphic.

A **terminal object** E is the range of a set of morphisms $S_n \to E$. Terminal objects subsume all other objects in the category in that all other objects S_n can be morphed into the terminal object, but the terminal object can not be morphed into any other object. Similar to initials, should another terminal exist, the two terminal are isomorphic in that they both have the same number of incoming morphisms and no outgoing morphisms (save the identity).



In order to quantify the *amount* of values denoted by a type, every mm-ADT type has an associated **quantifier** $q \in Q$ written `{q}` in `mmlang`, where Q is the carrier of an ordered algebraic ring with unity (e.g. integers $\mathbb{Z}$, reals in $\mathbb{R}$, $\mathbb{R}^2$, $\mathbb{R}^3$, ..., $\mathbb{R}^n$, unitary matrices, etc.).

Typically, integer quantifiers signify "amount." However, other quantifiers such as unitary matrices used in the representation of a quantum wave function, "amount" is a less accurate description as `objs` interact with constructive and destructive interference. Even in $\mathbb{Z}$, negative integers are possible and are leveraged for computing lazy set operations as demonstrated by intersection in the associated example.

```
mmlang> [[5,6,7],[7,5]{-1}]
==>6
```

The default **quantifier ring** of the mm-ADT VM is

$$(\mathbb{Z} \times \mathbb{Z}, +, *),$$

where $(0, 0)$ is the additive identity and $(1, 1)$ is the multiplicative identity (unity). The $+$ and $*$ binary operators

perform pairwise integer addition and multiplication, respectively. In `mmlang` if an `obj` quantifier is not displayed, then the quantifier is assumed to be the unity of the ring, or `{1,1}` in this case. Moreover, if a single value is provided, it is assumed to be repeated, where `{n}` is shorthand for `{n,n}`. Thus,

$$\texttt{int} \equiv \texttt{int\{1\}} \equiv \texttt{int\{1,1\}}.$$

One particular quantifier of every ring serves an important role in mm-ADT as both the additive identity and multiplicative annihilator — `{0}`. All `obj`s quantified with the respective quantifier ring's annihilator are non-terminal initial objects as exemplified in the adjoining example.

```
mmlang> _{0}[start,6]
==>6
mmlang> 6[is>7]
mmlang> 6[is>8]
mmlang> 6[is>9]
```

Types such as `int{0}` and `int{0}<=int[is,false]` are equivalent due to their quantifiers both being `{0}`. Throughout the documentation, all zero quantified `obj`s will be referred to as `_{0}`, `{0}`, or $\mathbf{0}$ (the **zero object**).

```
mmlang> 6{0}
mmlang> int{0}[plus,2]
mmlang> int[plus,2]{0}
mmlang> _{0}
```

Quantifiers serve an important role in type inference and determining, at compile time, the expected cost of a particular type definition (i.e., an instruction sequence). The table below itemizes common quantifier patterns that have a corresponding construction in other programming languages.

name	sugar	unsugared	description	mmlang example
some		`{1,1}`	a single `int`	`mmlang> int` `==>int`
option	`{?}`	`{0,1}`	0 or 1 `int`	`mmlang> int{?}<=int[is>0]` `==>int{?}<=int[is,bool<=int[gt,0]]`
none	`{0}`	`{0,0}`	0 `int`s	`mmlang> int{0}<=int[is,false]` `mmlang>`
exact	`{4}`	`{4,4}`	4 `int`s	`mmlang> int{4}<=int{2}[_,_]` `==>int{4}<=int{2}[id]{2}`
any	`{*}`	`{0,max}`	0 or more `int`s	`mmlang> int{*}<=rec{*}[get,'age',int]` `==>int{*}<=rec{*}[get,'age',int]`
given	`{+}`	`{1,max}`	1 or more `int`s	`mmlang> int{+}` `==>int{+}`

Types use quantifiers in two separate, but related, contexts: **type signatures** and **type definitions**.

Type Signature Quantification

A type signature's **domain** specifies the type and quantity of the `obj` required for evaluation. The **range** denotes what can be expected in return. `int{6}<=int{3}` states that given `3` `int`s, the type will return `6` `int`s. Quantifiers in a type signature are descriptive, used in type checking.

```
mmlang> 4 => int{6}<=int{3}[[plus,1],[plus,1]]
language error: int is not an int{3}
mmlang> 4{3} => int{6}<=int{3}[[plus,1],[plus,1]]
==>5{6}
mmlang> [4,5,6] => int{6}<=int{3}[[plus,1],[plus,1]]
==>5{2}
==>6{2}
==>7{2}
mmlang> [4{2},5{1},6{2}] => int{6}<=int{3}[[plus,1],[plus,1]]
language error: int{5} is not an int{3}
mmlang> [4{2},5{-1},6{2}] => int{6}<=int{3}[[plus,1],[plus,1]]
==>5{4}
==>6{-2}
==>7{4}
```

Much will be said about negative quantifiers. For now, note that negative quantifiers enable lazy, stream-based set theoretic operations such as intersection, union, difference, etc. Extending beyond integer quantification $(\mathbb{Z})$, negative quantifiers enable constructive and destructive interference in quantum computating $(\mathbb{C})$ and excitatory and inhibitory activations in neural computing $(\mathbb{R})$.

Type Definition Quantification

A type definition's **instructions** can be quantified. More specifically, a type's intermediate dtypes can be quantified. During type inference, the quantifier ring's $(+ / *)$-operators propagate the quantifiers through the types that compose the program.

```
mmlang> int{3}[[plus,1],[plus,1]]
==>int{6}<=int{3}[plus,1]{2}
mmlang> int{3}[plus,1]{2}
==>int{6}<=int{3}[plus,1]{2}
```

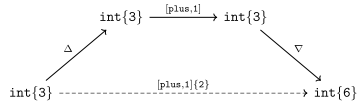
❶ ❷

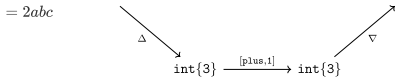
❶ Given `3` `int`s, `[plus,1]` will be evaluated (in parallel) twice. The result is `6` `int`s.

❷ The instruction `[plus,1]{2}` is the merging of two `[plus,1]` *branches*.

At type compilation, the branch optimizer "collapses" *type object* equivalent branches with no effect to the result. The branches' *type quantifiers* are added using the quantifier ring's $+$-operator (the quantifier group). Once collapsed, quantifiers can be moved left-or-right using the quantifier ring's multiplicative $*$-operator due to the *commutativity of quantifiers theorem* (the quantifier monoid). It is more efficient (especially as branches grow in complexity) to compute, for example, $2b$ than $b + b$. The example below demonstrates how type quantifiers are "collapsed" with $+$ and "slid" left (or right) with $*$.

$$\begin{aligned} a(b+b)c &= a(2b)c \\ &= a2bc \end{aligned}$$





The following two examples highlight the fact that type signature quantifiers are used for type checking and type definition quantifiers are used for type inference. The algebra of quantification will be explained in much more detail later when discussing the ring algebra of mm-ADT.

```
mmlang> 4{3} => [[plus,1],[plus,1]]
==>5{6}
mmlang> 4{3} => int{6}<=int{3}[[plus,1],[plus,1]]
==>5{6}
mmlang> 4{2} => int{6}<=int{3}[[plus,1],[plus,1]]
language error: int{2} is not an int{3}
```

$$\begin{aligned} \texttt{int\{q\}} &= 3 * (1 + 1) \\ &= (3 * 1) + (3 * 1) \\ &= 3 + 3 \\ &= 6 \end{aligned}$$

```
mmlang> 4{3} => [plus,1]{2}
==>5{6}
mmlang> 4{3} => int{6}<=int{3}[plus,1]{2}
==>5{6}
mmlang> 4{2} => int{6}<=int{3}[plus,1]{2}
language error: int{2} is not an int{3}
```

$$\begin{aligned} \texttt{int\{q\}} &= 3 * 2 \\ &= 6 \end{aligned}$$

Quantifier Commutativity

Each of these expressions is equivalent to `obj{0}`. This is demonstrated using the `;-poly` quantifier equation. `2*3*0 = 2*0*4 = 0*3*4`. In general, if there exists a **0**-quantified `obj` in a `obj` monoid expression, then the result is always `obj{0}`.

```
mmlang> 6{2}+{3}1+{0}2
mmlang>
mmlang> 6{2}+{0}1+{4}2
mmlang>
mmlang> 6{0}+{3}1+{4}2
mmlang>
```

All three expression evaluate to the same `9{24}` value. The quantifier ring has a commutative multiplicative monoid such that `2*3*4 = 3*4*2 = 4*2*1`.

```
mmlang> 6{2}+{3}1+{4}2
==>9{24}
mmlang> 6{3}+{4}1+{2}2
==>9{24}
mmlang> 6{4}+{2}1+{3}2
==>9{24}
```

If the quantifier ring is not commutative, it is still possible to propagate coefficients left or right through an `obj *`-expression. Regardless of the quantifiers being prime elements, quantifier propagation need not preserve the factors of a `*`. In this way, if the geometric sequence remains the same, any quantifier distribution is allowed.

```
mmlang> 6{2}+{3}1+{4}2
==>9{24}
mmlang> 6+{6}1+{4}2
==>9{24}
mmlang> 6+1+{24}2
==>9{24}
mmlang> 6+{12}1+{2}2
==>9{24}
mmlang> 6{6}+{2}1+{2}2
==>9{24}
```

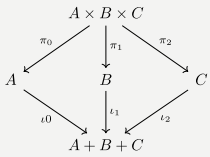
Quantifiers propagate along the the multiplicative `obj` monoid via their `*`-operator. They propagate along the additive `obj` group via their `+`-operator. In this way, if two branches have orthogonal quantifiers of the same magnitude, then when they leave the `+`-group to be additively merged onto the `*`-monoid, they cancel each other out. Various set theoretic and quantum operations make use of constructive and deconstructive quantifier interference when computing.

```
mmlang> 6[+{-1}1+{2}1,+{2}2]
mmlang>
mmlang> 6[+{-1}1+1,+2]{2}
mmlang>
mmlang> 6{2}[+{-1}1+1,+2]
mmlang>
```

Type Composition

Stream Ring Theory

Stream ring theory studies a particular type of algebraic ring constructed from a direct product of a **function** semiring and **coefficient** ring. Along with the standard ring axioms over $*$ and $+$, the theory requires that every stream ring uphold five additional axioms regarding coefficient dynamics. Categorically, every stream ring forms an additive category with biproducts. A biproduct has both projection (product) and injection (coproduct) morphisms that capture the splitting and merging of streams. Along with the *atemporal stream theorem* derived from the stream ring axioms, biproduct streams have practical significance in asynchronous distributed computing environments that primarily enjoy embarrassingly parallel processing, but where, at certain space and time synchronization points, data needs to be co-located by the *reduce* near-ring operator $\oplus$.



mm-ADT adopts the algebra of stream ring theory, but uses the term **instruction** for *function* and **quantifier** for *coefficient*. Moreover, mm-ADT extends stream ring theory with an inductive, dependent type theory based on a multi-sorted stream ring with interval quantifiers called the **type ringoid** whose free algebra is captured by the **type graph**.

The mm-ADT virtual machine has two distinct algebraic layers: the instruction set architecture and the stream ring. The instructions (`inst`) specify how input `objs` are mapped to output `objs` and has a graphical realization as a generalized Cayley graph and/or a commuting diagram. The `inst` algebra is evaluated by the processor-oriented stream ring algebra. A stream ring has three operators for constructing types: $*$, $+$, and $\oplus$, where the first two are the classic ring operators and the last is particular to a stream ring. The stream ring's multiplicative monoid's $*$-operator concatenates **serial streams**, the additive abelian group's $+$-operator composes **parallel streams**, and the stream near-ring's non-commutative group's $\oplus$-operator reduces streams down to a **singleton stream**.

$$\left[m_0 * m_1 * \ldots * m_n\right] \begin{bmatrix} g_0 \\ + \\ g_1 \\ + \\ \vdots \\ + \\ g_n \end{bmatrix} |\oplus r\rangle \left[* \ldots *\right] \begin{bmatrix} + \\ \vdots \\ + \end{bmatrix} \ldots$$

op	inst	sugar
$*$	`[juxt]`	`=>`
$+$	`[branch]`	`=[`
$\oplus$	`[barrier]`	`=|`

The illustration above is an intuitive visualization of an mm-ADT type from the perspective of monoidal, group, and near-ring magmas interacting with one another in a series ($*$) of expansions ($+$) and contractions ($\oplus$), where $m_i, g_i, r \in$ `obj`. These three stream operators have a corresponding realization in `inst` as higher-order instructions. It is through these instructions that the other instructions are grounded in the underlying stream ring algebra of the mm-ADT VM.

op	inst	sugar

The illustration above is an intuitive visualization of an mm-ADT type from the perspective of monoidal, group, and near-ring magmas interacting with one another in a series ($*$) of

Stream Ring Operators

An `obj` is defined

$$\texttt{obj} = (\texttt{type} \times \texttt{q}) + (\texttt{value} \times \texttt{q}).$$

Thus, an `obj` is either a **quantified type** or a **quantifier value**. With respects to the stream ring's three operators, there are 4 binary `obj` configurations (2^2) that each operator will encounter: **value**/**value**, **value**/**type**, **type**/**value**, and **type**/**type**. The table below presents the equations of each operator for each of the 4 configurations, where it is through operations involving right-hand side (RHS) types that the instructions in `inst` are applied to `objs` (e.g., $t(v)$ is evaluation and $t(t)$ is compilation). Thus, the stream ring is the fundamental algebra of the mm-ADT VM.

In the examples to follow, a term of the form x_q is a type (t) or value (v) with quantifier q and $\mathbf{x}$ is a stream of types or values, where an $i \in \mathbb{N}$ as in $\mathbf{x}_i$ denotes the i^{th} `obj` of $\mathbf{x}$.

·	[juxt] =>	[branch] =[	[barrier] =|
value · value	$v_{0_q} \Rightarrow v_{1_r} = v_{1_{q*r}}$	$[v_{0_q}, v_{1_r}] = \begin{cases} v_{0_{q+r}} & \text{if } v_0 == v_1 \\ [v_{0_q}, v_{1_r}] & \text{otherwise.} \end{cases}$	$\mathbf{v} \dashv v_{1_r} = v_{1_r}$
value · type	$v_q \Rightarrow t_r = t(v)_{q*r}$	$[v_q, t_r] = [v_q, t_r]$	$\mathbf{v} \dashv t_r = \left(\bigoplus_{i=0}^{n} t(\mathbf{v}_i) \right)_r$
type · value	$t_q \Rightarrow v_r = v_{q*r}$	$[t_q, v_r] = [t_q, v_r]$	$\mathbf{t} \dashv v_r = v_r$
type · type	$t_{0_q} \Rightarrow t_{1_r} = t_1(t_0)_{q*r}$	$[t_{0_q}, t_{1_r}] = \begin{cases} t_{0_{q+r}} & \text{if } t_0 == t_1 \\ [t_{0_q}, t_{1_r}] & \text{otherwise.} \end{cases}$	$\mathbf{t_0} \dashv t_1 = \left(\bigoplus_{i=0}^{n} t_1(\mathbf{t}_{0_i}) \right)_r$

The simple example program below uses all 3 stream ring operators. The subsequent table provides a detailed decomposition of the evaluation. The instruction `[model,mmx]` loads the mm-ADT extension model that contains common `mm` base type coercions such as `int<=str`, `real<=int`, etc.

```
mmlang> :[model,mmx]
==>_
mmlang> 4 => int[mult,2] => str[plus,'0'] => real
==>80.0
mmlang> 4 => int[mult,2] =[ int=>str[plus,'0'],int=>str[plus,'.1'] ] => real
==>80.0
==>8.1
```

type	4 => int[mult,2]	=[str[plus,'0'],str[plus,'.1']]	=> real	=|[plus,x]
d/r	int<=int	str{2}<=int	real{2}<=str{2}	real<=real{2}
value	4=>8	8=>['80','8.1']	['80','8.1']=>[80.0,8.1]	[80.0,8.1]=>88.1
q	{1}=>{1}	{1}=>{2}	{2}=>{2}	{2}=>{1}
op	$*$ monoid	$+$ abelian group	$*$ monoid	$\oplus$ group
term	series	expansion	series	contraction

Stream Ring Algebra

Ringoids

An algebraic ring $(A, +, *, \mathbf{0}, \mathbf{1})$ is composed of an additive abelian group $(A, +, \mathbf{0})$ and a multiplicative monoid $(A, *, \mathbf{1})$ that share the same carrier set A and whose operators are bound by the axiom of distributivity that requires

$$a * (b + c) = ab + ac$$
$$(a + b) * c = ac + bc.$$

A ringoid generalizes a ring with a multi-sorted carrier $A = (A_0, A_1, ..., A_n)$ such that the magmas of the binary operators are partial functions lacking closure. In other words, a ringoid is a ring with a type system with the consequence that for any element $a \in A_i$ and $b \in A_j$, it is not required that $a + b$ nor $a * b$ be defined.

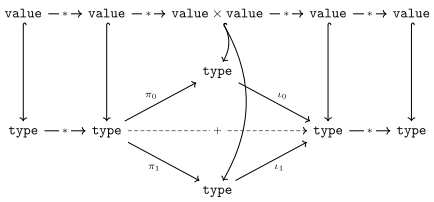
For all the intended practical uses for mm-ADT, the instruction set architecture constrains the stream ring to a stream ringoid. However, given that the ISA is predicated on the **model**, the more general and more widely known *ring* will serve as the classifying term.

The mm-ADT stream ring is the algebraic ring

$$(\texttt{obj}, +, *, \oplus, \mathbf{0}, \mathbf{1}),$$

where

- `obj` is the carrier set containing all quantified objects,
- $+$ the additive *parallel branch* operator,
- $*$ the multiplicative *serial chain* operator,
- $\oplus$ is the reducing *barrier* operator,
- $\mathbf{0}$ the additive identity, and
- $\mathbf{1}$ the multiplicative identity.



The equation `obj` $=$ `type` $+$ `value`) and the suggestive illustration above highlight two important uses of the ring's multiplicative binary $*$-operator:

1. $*:\texttt{type}\times\texttt{type}\to\texttt{type}$ generate functions graph (**program compilation**) and,
2. $*:\texttt{value}\times\texttt{type}\to\texttt{value}$ *stream* values through the type structure (**program evaluation**).

Along with the standard ring axioms (save operator closure), the `obj` stream ring respects the five additional axioms of **stream ring theory**. The following tables provide a consolidated summary of the ring axioms, stream ring axioms and their realization in mm-ADT via examples in `mmlang` using both `obj` **values** and **types**.

The `mmlang` examples are rife with syntactic sugars. The term `_{0}` (sugar'd `{0}`) is $\mathbf{0}$, `_{1}` (sugar'd `{1}`) is $\mathbf{1}$, `[a,b,c]` denotes `[branch,(a,b,c)]` and `+{q}n` denotes `[plus,n]{q}`. Finally, while `[,]` and `[;]` are defined as binary operators, due to the associativity axioms of the respective additive group and multiplicative monoid of a ring, `[,]` and `[;]` are effectively n-ary operators and will be used as such in examples to follow.

Ring Axioms

Axioms are the "hardcoded" equations of a system. Regardless of any other behaviors the system may express, if the system always respects the ring axioms, then the system is (in part) a ring.

axiom	equation	mmlang values	mmlang types
Additive Abelian Group—$(\texttt{obj},\ +,\ \mathbf{0})$			

Additive associativity	$(a+b)+c$ $=a+(b+c)$	`mm> [['a','b'],'c']`
		`==>'a'`
		`==>'b'`
		`==>'c'`
		`mm> ['a',['b','c']]`
		`==>'a'`
		`==>'b'`
		`==>'c'`
			`mm> str[[[id],[id]],[id]]`
			`==>str{3}<=str[id]{3}`
			`mm> str[[id],[[id],[id]]]`
			`==>str{3}<=str[id]{3}`

Additive commutativity	$a+b$ $=b+a$	`mm> ['a','b']`
		`==>'a'`
		`==>'b'`
		`mm> ['b','a']`
		`==>'b'`
		`==>'a'`
			`mm> str[[id]{2},[id]{3}]`
			`==>str{5}<=str[id]{5}`
			`mm> str[[id]{3},[id]{2}]`
			`==>str{5}<=str[id]{5}`

Additive identity	$a+\mathbf{0}=a$	`mm> ['a',{0}]` `==>'a'`	`mm> str[[id],{0}]` `==>str`
Additive inverse	$a+(-a)=\mathbf{0}$	`mm> ['a','a'{-1}]` `mm>`	`mm> str[[id],[id]{-1}]` `mm>`
Multiplicative Monoid—$(\texttt{obj},\ *,\ \mathbf{1})$			

Multiplicative associativity	$(a\cdot b)\cdot c$ $=a\cdot(b\cdot c)$	`mm> [['a';'b';'c']`
		`==>'c'`
		`mm> ['a';['b';'c']]`
		`==>'c'`
			`mm> str[[[id];[id]];[id]]`
			`==>str`
			`mm> str[[id];[[id];[id]]]`
			`==>str`

Multiplicative identity	$a\cdot\mathbf{1}=a$	`mm> ['a';{1}]` `==>'a'`	`mm> str[[id];{1}]` `==>str`
Ring with Unity—$(\texttt{obj},\ +,\ *,\ \mathbf{0},\ \mathbf{1})$			

Left distributivity	$a\cdot(b+c)$ $=ab+ac$	`mm> ['a';['b','c']]`
		`==>'b'`
		`==>'c'`
		`mm> [['a';'b'],['a';'c']]`
		`==>'b'`
		`==>'c'`
			`mm> str[[id];[[id],[id]]]`
			`==>str{2}<=str[id]{2}`
			`mm> str[[[id];[id]],[[id];[id]]]`
			`==>str{2}<=str[id]{2}`

Right distributivity	$(a+b)\cdot c$ $=ac+bc$	`mm> [['a','b'];'c']`
		`==>'c'{2}`
		`mm> [['a';'c'],['b';'c']]`
		`==>'c'{2}`
			`mm> str[[[id],[id]];[id]]`
			`==>str{2}<=str[id]{2}`
			`mm> str[[[id];[id]],[[id];[id]]]`
			`==>str{2}<=str[id]{2}`

Ring Theorems

The axioms of a theory entail its theorems. Stated in reverse, theorems are the derivations of an axiomatic system. Once a system is determined to be a ring, then all the theorems that have been proved about rings in general are also true for that system.

theorem	equation	mmlang values	mmlang types
Ring with Unity—$(\texttt{obj},\ +,\ *,\ \mathbf{0},\ \mathbf{1})$			
Additive factoring	$a+b=a+c$ $\Rightarrow b=c$		
Unique factoring	$a+b=\mathbf{0}$ $\Rightarrow a=-b$ $\Rightarrow b=-a$		

Inverse distributivity	$-(a+b)$ $=(-a)+(-b)$	`mm> ['a','b']{-1}`
		`==>'a'{-1}`
		`==>'b'{-1}`
		`mm> ['a'{-1},'b'{-1}]`
		`==>'a'{-1}`
		`==>'b'{-1}`
			`mm> str[[id],[id]]{-1}`
			`==>str{-2}<=str[id]{-2}`
			`mm> str[[id]{-1},[id]{-1}]`
			`==>str{-2}<=str[id]{-2}`

Inverse distributivity	$-(-a)=a$	`mm> ['a'{-1}]{-1}` `==>'a'`	`mm> str[[id]{-1}]{-1}` `==>str`

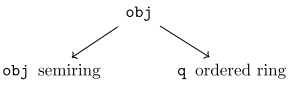
theorem	equation	mmlang values	mmlang types

Annihilator	$a*\mathbf{0}$ $=\mathbf{0}$ $=\mathbf{0}*a$	`mm> ['a';{0}]`
		`mm>`
		`mm> [{0};'a']`
		`mm>`
			`mm> str[[id];{0}]`
			`mm>`
			`mm> str[{0};[id]]`
			`mm>`

Factoring	$a*(-b)$ $=-a*b$ $=-(a*b)$	`mm> ['a';'b'{-1}]`
		`==>'b'{-1}`
		`mm> ['a'{-1};'b']`
		`==>'b'{-1}`
		`mm> ['a';'b']{-1}`
		`==>'b'{-1}`
			`mm> str[[id];[id]{-1}]`
			`==>str{-1}<=str[id]{-1}`
			`mm> str[[id]{-1};[id]]`
			`==>str{-1}<=str[id]{-1}`
			`mm> str[[id];[id]]{-1}`
			`==>str{-1}<=str[id]{-1}`

Factoring	$(-a)*(-b)$ $=a*b$	`mm> ['a'{-1};'b'{-1}]`
		`==>'b'`
		`mm> ['a';'b']`
		`==>'b'`
			`mm> str[[id]{-1};[id]{-1}]`
			`==>str`
			`mm> str[[id];[id]]`
			`==>str`

Stream Ring Axioms



Stream ring theory studies *quantified objects*. The quantifiers must be elements of an ordered ring with unity. The stream ring axioms are primarily concerned with quantifier equations and their relationship to efficient stream computing. The most common quantifier ring is integer pairs (denoting a range) with standard pairwise addition and multiplication, $(\mathbb{Z}\times\mathbb{Z},\ +\ ,\ *\ ,(0,0),(1,1))$. However, the theory holds as long as the quantifiers respect the ring axioms and, when coupled to an object, they respect the stream ring axioms.

The algebra underlying most type theories operate as a semiring(oid), where the additive component is a monoid as opposed to an invertible group. In mm-ADT, the elements of the additive component can be inverted by their corresponding *negative type* (or negative `obj` in general). Thus, mm-ADT realizes an additive groupoid, where, for example, the `,-poly [int{1},int{-1}]` is `int{0}` which is isomorphic to the initial `obj{0}`.

axiom	equation	mmlang values	mmlang types
Bulking	$xa+ya$ $=(x+y)a$	`mm> ['a'{2},'a'{3}]` `==>'a'{5}`	`mm> str[[id]{2},[id]{3}]` `==>str{5}<=str[id]{5}`
Applying	$xa*yb=(xy)ab$	`mm> 'a'{2}['b'{3}]` `==>'b'{3}`	`mm> str{2}[[id]{3}]` `==>str{6}<=str{2}[id]{3}`

Splitting	$xa*(yb+zc)$ $=(xy)ab+(xz)ac$	`mm> 'a'{2}['b'{3},'c'{4}]`
		`==>'b'{3}`
		`==>'c'{4}`
		`mm> ['b'{6},'c'{8}]`
		`==>'b'{6}`
		`==>'c'{8}`
			`mm> str{2}[[id]{3},[id]{4}]`
			`==>str{14}<=str{2}[id]{7}`
			`mm> str[[id]{6},[id]{8}]`
			`==>str{14}<=str[id]{14}`

Merging	$((xa)+(yb))$ $=(xa+yb)$	`mm> [['a'{2}],['b'{3}]]`
		`==>'a'{2}`
		`==>'b'{3}`
		`mm> ['a'{2},'b'{3}]`
		`==>'a'{2}`
		`==>'b'{3}`
			`mm> str[[[id]{2}],[[id]{3}]]`
			`==>str{5}<=str[id]{5}`
			`mm> str[[id]{2},[id]{3}]`
			`==>str{5}<=str[id]{5}`

Removing	$(\mathbf{0}a+b)=b$	`mm> ['a'{0},'b']` `==>'b'`	`mm> str[{0},[id]]` `==>str`

Type System

Constructive Type Theory

Classical type theory associates **types** with **sets**, where the type A denotes a set containing all the elements of the type. The validity of the type assignment $a:A$ is determined via set membership of whether $a\in A$. This interpretation of a type implies that all elements exist in some Platonic world (as types can have an infinite number of elements).
Constructive type theory, on the other hand, realizes types as *generators* of their elements, where a type's definition specifies the rules for constructing objects of that type. Constructions imply functions as opposed to sets.

mm-ADT's type theory is a constructive type theory. mm-ADT types are not sets, but instead are functions that generate objects of the type (**range**) from objects of another type (**domain**). Thus, the determination of whether a is typed A is a determination of whether the function $A(a)$ is defined at a. Such determinations are called *evaluations*.

The table below itemizes the judgements of Martin-Löf type theory (a constructive type theory) alongside mm-ADT's corresponding construct.

Judgement	Martin-Löf type	mm-ADT
Type declaration	A Type	`A`
Type assignment	$a:A$	`A:a` or `a=>A`
Type equality	$A=B$	`B<=A`
Type dependency	$(x:A)B$	`B<=A`
Type substitution	$B[x/a]$	`a=>B`
Σ type (coproduct)	$A\times B$	`(A;B)`
Π type (product)	$A\to B$	`B<=A`

- **Type Dependency and Π Types**: The structure of an mm-ADT type maintains both the type's signature (`B<=A`) and the type's definition `[f]`). The form of a typical type dependency is `B<=A[f]` , where a `B` is *constructed* from an `A` by applying instruction `[f]` . In standard mathematical notation, `B<=A[f]` is denoted $f:A\to B$ which is the named

version of the Π type $A \to B$ and therefore, a type dependency and a Π type are the same construct in mm-ADT.

- **Type Equality**: Note that type equality, dependency and Π all have the same form in mm-ADT. However, while type equality is a type dependency and a Π type, every type dependency and Π type is not a type equality. The type `B<=A[f]` is not a type equality. The type equality $A = B$ is an identity function `B<=A[noop]`, where a `B` is realized by doing nothing to an `A`. The type `B<=A[noop]` is more succinctly written `B<=A` and thus, `B` is not *constructed* from `A`, but is *equivalent* to `A` and every `A` can be typed `B` as is.
- Σ **Types**: Contrary to their presentation in the table above, Σ types are more general than standard products. Σ types are *dependent products*. As an example, the "ascending pair" Σ type is defined

$$\texttt{apair} = \sum_{m:\mathbb{Z}} \sum_{n:\mathbb{Z}} m < n,$$

where the two integers $m, n \in \mathbb{Z}$ are the components of the product and m is smaller than n ($m < n$). This definition specifies a relation between the components of the product (i.e., a dependence) that must be satisfied by every object of this type. In `mmlang`, `apair` is written

```
apair<=(int:m;int:n)[is,m<n]
```

```
mmlang> :[model,mm]
        [define,apair<=(int:m;int:n)[is,m<n]]
==>_[define,apair<=(int<m>;int<n>)[is,bool<=(int<m>;int<n>)<.m>[lt,n]]]
mmlang> (1;2)   => apair
==>apair:(1;2)
mmlang> (2;1)   => apair
language error: (2;1) is not an apair
mmlang> (1;2;3) => apair
language error: (1;2;3) is not an apair
mmlang> 1       => apair
language error: 1 is not an apair
```

mm-ADT's type system is founded on 3 classes of ctypes: **anonymous**, **mono**, and **poly** types. Within the `mono` and `poly` types, further subdivisions exist. These foundational types are the building blocks by which all other types are constructed using the ring operators of mm-ADT's stream ring algebra. At the limit, an mm-ADT program is best understood as a "complex" type.

Anonymous Types

The type `bool<=int[gt,10]` has a **range** of `bool` and a **domain** of `int`. When the type is written without it's range as `int[gt,10]`, the range is deduced. The `int` domain ctype is applied to `[gt,10]` to yield a `bool`. A type with an unspecified range is called an an **anonymous type** and is denoted `_` in `mmlang` (or with no character in many situations). An anonymous range is the result of an anonymous domain.

`_` range	`_` domain

`mmlang> bool<=int[gt,10]` ❶
`==>bool<=int[gt,10]`
`mmlang> _<=int[gt,10]` ❷
`==>bool<=int[gt,10]`
`mmlang> int[gt,10]` ❸
`==>bool<=int[gt,10]`
`mmlang> _`
`==>_`
	`mmlang> int{?}<=int[is,bool<=int[gt,10]]` ❶
	`==>int{?}<=int[is,bool<=int[gt,10]]`
	`mmlang> int[is,int[gt,10]]` ❷
	`==>int{?}<=int[is,bool<=int[gt,10]]`
	`mmlang> int[is,_[gt,10]]` ❸
	`==>int{?}<=int[is,bool<=int[gt,10]]`
	`mmlang> int[is,[gt,10]]` ❹
	`==>int{?}<=int[is,bool<=int[gt,10]]`

❶ The domain and range of the type are fully specified. ❷ A type with a specified domain of `int` and a specified range of `_`. ❸ An `mmlang` sugar where if no range is specified, and it differs from the domain, then `_` is assumed.	❶ The domain and range of the outer and inner nested type are fully specified. ❷ The nested type has a specified domain of `int`. ❸ The nested type has an unspecified domain of `_`. ❹ An `mmlang` sugar where if no domain is specified, `_` is assumed.

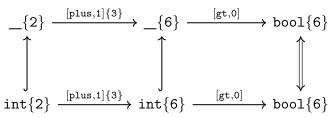
In the **anonymous type** `_{2}[plus,1]{3}[gt,0]`, the `[plus]` instruction is polymorphic, but `[gt]` isn't. Thus, the anonymous type's range can be deduced, but without a domain ctype, the internal dtype is also unknown. When `int{2}` is provided, it triggers a cascade of deductions that determines incident types of the instructions in the `obj` graph. The quantifier ring, when moving "horizontally" across the multiplicative `inst` monoid alters quantifiers using its multiplication operator, where $2 * 3 * 1 = 6$. Thus, `bool{6}<=int{2}[plus,1]{3}[gt,0]`

```
mmlang> _{2}[plus,1]{3}[gt,0][explain]
==>'
bool{6}<=_{2}[plus,1]{3}[gt,0]

inst            domain       range     state
-----------------------------------------------
[plus,1]{3}     _{2}     =>  _{6}
[gt,0]          _{6}     =>  bool{6}
'
```

```
mmlang> int{2}[plus,1]{3}[gt,0][explain]
==>'
bool{6}<=int{2}[plus,1]{3}[gt,0]

inst            domain         range     state
-------------------------------------------------
[plus,1]{3}     int{2}     =>  int{6}
[gt,0]          int{6}     =>  bool{6}
'
```



Anonymous Type Uses

Anonymous types are useful in other situations besides lazy typing expressions.

```
mmlang> 5-<(_,_)                                  ❶
==>(5{2})
mmlang> -5[is>0 -> +2 | _ -> +10]                 ❷
==>5
mmlang> 5-<([a,int],[a,_],[a,str])                ❸
==>(true{2},false)
```

❶ When no processing is needed on a split, `_` should be provided.

❷ When used in a `|-rec poly`, `_` is used to denote the default case.

❸ `5` is both an `int` and a `_`, but not a `str`.

In general, anonymous types are wildcards because they pattern match to every `obj`. As will be demonstrated soon,

when a **variable** is specified (e.g. `[plus,x]`) or a new type is specified (e.g. `x:42`), The `x` is a **named anonymous type**. The entailment of this is that types and variables are in the same namespace. Two presumably self-explanatory examples are provided below with a more detailed discussion of variables and named types forthcoming.

variables

```
mmlang> 1 => int[plus,2][to,x][plus,3][mult,x]
==>18
mmlang> int[plus,2][to,x][plus,3][mult,x][explain]
==>'
int[plus,2]<x>[plus,3][mult,x]

inst        domain      range state
-------------------------------------
[plus,2]    int    =>   int
[plus,3]    int    =>   int    x->int
[mult,x]    int    =>   int    x->int
'
```

named types

```
mmlang> 1 => int[define,x<=int[plus,2]][as,x]
==>x:5
mmlang> int[define,x<=int[plus,2]][as,x][explain]
==>'
x<=int[define,x<=int[plus,2]][as,x<=int[plus,2]]

inst                        domain      range state
-----------------------------------------------------
[define,x<=int[plus,2]]     int    =>   int    x->x<=int[plus,2]
 [plus,2]                    int    =>    int   x->x<=int[plus,2]
[as,x<=int[plus,2]]         int    =>   x      x->x<=int[plus,2]
 [plus,2]                    int    =>    int   x->x<=int[plus,2]
'
```

Mono Types

The mm-ADT type system can be partitioned into **mono types** (monomials) and **poly types** (polynomials). There are 4 mono types, each denoting a classical primtive data type: `bool`, `int`, `real`, and `str`. The associated table presents the typical operators (sugared instructions) that can be applied to each mono type. The table also includes their respective additive $(\mathbf{0})$ and multiplicative $(\mathbf{1})$ identities.

type	inst	0	1
bool	&& || - !	false	true
int	* + - > < >= =<	0	1
real	* + - > < >= =<	0.0	1.0
str	+ > < >= =<	''	

A few of the more interesting aspects of the mono types are detailed in the following subsections.

Zero and One

The instructions `[zero]` and `[one]` are constant polymorphic instructions. Each provides a unique singleton value associated with the type of the respective incoming `obj`. In the example to follow, `=` is sugar for pairwise `[combine]`, with round robin evaluation for overflow. As examples, `(a,b,c)=(x,y)` yields `(ax,by,cx)` and `(a,b,c)=(x)` is `(ax,bx,cx)` (i.e.,right scalar multiplication).

```
mmlang> (true;6;5.5;'ryan')=([zero])
==>(false;0;0.0;'')
mmlang> (true;6;5.5)=([one])
==>(true;1;1.0)
mmlang> 'ryan'[one]
language error: 'ryan' is not supported by [one]
```

Each type's $\mathbf{0}$ and $\mathbf{1}$ value serves as the `[plus]` and `[mult]` instruction identities, respectively. Furthermore, for types respecting a ring with unity algebra, `[zero]` is their corresponding multiplicative annihilator.

```
mmlang> (true;6;5.5;'ryan')=(+[zero])
==>(true;6;5.5;'ryan')
mmlang> (true;6;5.5)=(*[one])
==>(true;6;5.5)
mmlang> (true;6;5.5)=(*[zero])
==>(false;0;0.0)
```

Poly Types

Free Objects

A magma algebra is defined by a carrier set A along with a binary operator $\cdot : A \times A \to A$ that combines any two A-elements into one, where as an example, if $a, b, c \in A$, then $a \cdot b \mapsto c$ (with $ab = c$ being a more concise encoding). Implicit in the binary operator behavior is a set of axioms denoting "hardcoded" A-related equations that a structure must obey should it be an instance of the $(A, \cdot)$ algebra.

Given another element $d \in A$ such that $ad = c$, then it is unknown whether ab or ad was used to derive c. Assuming the general case that all elements do not have unique two element factors in A, then like the logical operations of `AND` and `OR`, the binary operator $\cdot$ is an irreversible, lossy operation. This can be remedied in a number of ways. The more efficient, general solution is a sequence of $\cdot$-compositions stored in a list. Such structures are A-"programs" that can be executed against *any* A-machine. If $a, b, c, d, e \in A$, then an example A-program is

$$aabbbcadebdcaecadeeeeabccbcaabb.$$

While the individual elements of the A-program are in A, the program as a whole is a *single* element in A^*. A^* is the infinite set of all possible A-element sequences of arbitrary length called the Kleene closure over A. From this vantage point, the elements of A are called **letters** and the elements of A^* are called **words**. The set A^* is the carrier set of another algebra $(A^*, \circ)$, where $\circ : A^* \times A^* \to A^*$ concatenates two words into a single word (i.e. list concatenation). This algebra is used to "code" A-programs. In the world of abstract algebra, this new $(A^*, \circ)$ algebra is called the free algebra over A.

A word in A^* can be reduced to a single letter in A via a homomorphism that relates $(A^*, \circ)$ and $(A, \cdot)$ denoted $\eta : A^* \to A$. Thus, given any A^*-program, η "executes" the program on some A-machine. If the η-mapping is preserved, then the answer to whether c was arrived at via ab or ad is known. mm-ADT preserves such mappings in a structure known as the `obj` graph. mm-ADT's graph-encoding of a free algebraic trace is the foundation of numerous mm-ADT capabilities including abstract interpretation, program state, metaprogramming, and reversible computing.

A `poly` is a free object. Free objects are universal structures in that they respect the equations of an abstract algebra, but not the equations of an instance of the abstract algebra. Hence the term *free* as in "free" from constraint of the concrete—i.e., universal. Examples of these two classes of equations are provided in the table below. If the concrete algebra equations appear random, it is because they are. Each concrete algebra's operator(s) map elements-to-elements in a manner specific to an application domain and as such, are not *universal* equations.

	abstract algebra equations	concrete algebra equations
identity	$a \cdot \mathbf{1} = a$	$a \cdot b = b$
inverses	$a \cdot a^{-1} = \mathbf{1}$	$a \cdot b^{-1} = \mathbf{1}$
associativity	$a \cdot (b \cdot c) = (a \cdot b) \cdot c$	$(a \cdot b) \cdot c = b \cdot c$
commutativity	$a \cdot b = b \cdot a$	$a \cdot a = a$

In mm-ADT, `polys` are

1. **collection** data structures, where the collection's semantics are tied to the operator(s) of the free object's abstract algebra, and
2. **control** data processes, where the control semantics are likewise a consequence of the underlying free object's algebra.

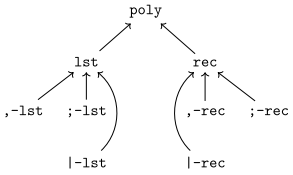
The mm-ADT `poly` is versatile because it is agnostic to the types and values contained therein while remaining in one-to-one correspondence with the stream ring algebra's operators' axioms and entailed theorems. More poetically, `poly` is the bedrock upon which the mm-ADT algebraic ecology is sustained.

Poly Types and Values

There are **type** `polys` and there are **value** `polys`. A type `poly` contains at least one type `obj` and is typically intended for use as a flow control structure. A value `poly` is composed of only value `objs` with a common use as a collection data structure.

As a practical consideration, mm-ADT offers two kinds of `poly`: `lst` (list) and `rec` (record), where theoretically, `rec` is simply a `lst` with some added conveniences that make typical programming patterns easier to express. Finally, a `poly` is associated with one of three algebras that comprise mm-ADT's stream ring: `,` (abelian group), `;` (monoid), or `|` (near-ring).

$$\texttt{poly} = \texttt{lst} \qquad\qquad +\texttt{rec}$$
$$\texttt{poly} = (\texttt{,-lst} + \texttt{|-lst} + \texttt{;-lst}) + (\texttt{,-rec} + \texttt{|-rec} + \texttt{;-rec}).$$



poly	sep	access	value	type
lst	,	all	multiset	union
	;	last	list	chain
	|	head	scalar	coalesce
rec	,	all match	multimap	cascade
	;	last match	record	condition
	|	first match	pair	switch

There are three instructions that are of primary importance for `poly` with one being a composite of the other two.

1. `[split]` (`-<`): propagates an incoming `obj` through a `poly` according to its algebra.
2. `[merge]` (`>-`): aggregates outgoing `objs` from a `poly` according to its algebra.
3. `[branch]` (`=[`): propagates and then aggregates `objs` through a `poly` according to its algebra.

The details of these instructions will be discussed in full over the following subsections.

Deep Dive 2. Mathematical Notation Conventions for mm-ADT

The `mmlang` notation is able to denote every possible type through the `obj` graph. That is, is can be used to define automata that can recognize every possible path through the `obj` graph. A companion notation exists separate from `mmlang` which is more aligned with the formalisms used in standard mathematical treatments of an algebraic system. This notation is called **mm notation**.

mmlang	mmnotation	
=>	$*$	$\prod$
=[	$+$	$\coprod$
=|	$\oplus$	$\bigoplus$

- **Monoidal composition ($*$)**: A non-branch serial path through the `obj` graph is expressed with the multiplicative $\prod$ operator. Given sequence of types $\mathbf{a} = [a_0, a_1, ..., a_n]$, their monoidal composition is denoted

$$\prod_{i=0}^{n} \mathbf{a}_i \quad \text{or} \quad \mathbf{a}_*$$

  where, for the latter, it is implied that entire sequence is left-to-right composed (via fold) using the stream ring multiplicative $*$-operator. While the first representation is more verbose, it's use will become apparent when considering intermediate operations within a monoidal composition.

- **Groupoidal composition ($+$)**: Parallel paths through the `obj` graph is denoted with the respective additive operator. Thus, given n-independent types $\mathbf{a} = [a_0, a_1, ..., a_n]$, their parallel evaluation is denoted

$$\coprod_{i=0}^{n} \mathbf{a}_i \quad \text{or} \quad \mathbf{a}_+$$

- **Near-ring reduction ($\oplus$)**: A path barrier is denoted with the $\oplus$ reduction operator. Given n-types (serial or parallel), their reduction using using the function $f: A^* \to B$ is denoted

$$\bigoplus_{i=0}^{n} f(\mathbf{a}_i) \quad \text{or} \quad \mathbf{a}_{\oplus f}$$

It is important to note that the composition operator on the vector representation is portrayed bottom right. The reason being is that regular language syntax is reserved for the top right. Next, in order to apply the same m-serial types to n-parallel types the additive and multiplicative operators are used in conjunction.

$$\begin{aligned}\coprod_{i=0}^{n} \mathbf{a}_i \coprod_{j=0}^{m} \mathbf{b}_j &= (\mathbf{a}\mathbf{b}_*)_+ \\ &= (a_0\mathbf{b}_* + a_1\mathbf{b}_* + \ldots + a_n\mathbf{b}_*) \\ &= (a_0 b_0 b_1 \ldots b_m) + \ldots + (a_n b_0 b_1 \ldots b_m)\end{aligned}$$

The additive/multiplicative pattern emphasizes the monoid ring interpretation of mm-ADT as well as the well know ring property that every ring is isomorphic to the composition of endomorphisms ($*$) of the ring's additive abelian group ($+$), where each a_i group element has applied a series of $\mathbf{b}$ morphisms back to the carrier set (i.e., an endomorphism).

Poly Collections

A `poly` can be used a collection data structure. There are a total of 6 sorts of collections as there are two kinds of `poly` (`lst` and `rec`) and each kind has 3 algebras (`,` , `;` , `|`).

lst	collection	mmlang example	rec	collection	mmlang example

`,`	abelian group
	multiset
		`mm> ('a'{?},'b','a','b')`
		`==>('a'{1,2},'b'{2})`
		`mm> ('a'{?},'b','a','b')>-`
		`==>'a'{1,2}`
		`==>'b'{2}`
			`,`	abelian group
				multimap
					`mm> ('a'->1,'b'->2,'a'->3)`
					`==>('a'->[1,3],'b'->2)`
					`mm> ('a'->1,'b'->2,'a'->3)>-`
					`==>1`
					`==>3`
					`==>2`

`;`	monoid
	list
		`mm> ('a';'b';'a';'b')`
		`==>('a';'b';'a';'b')`
		`mm> ('a';'b';'a';'b')>-`
		`==>'b'`
			`;`	monoid
				record
					`mm> ('a'->1;'b'->2;'a'->3)`
					`==>('a'->1;'b'->2;'a'->3)`
					`mm> ('a'->1;'b'->2;'a'->3)>-`
					`==>3`

`|`	near-ring
	scalar
		`mm> ('a'|'b'|'a'|'b')`
		`==>('a'|'b')`
		`mm> ('a'|'b'|'a'|'b')>-`
		`==>'a'`
			`|`	near-ring
				pair
					`mm> ('a'->1|'b'->2|'a'->3)`
					`==>('a'->1)`
					`mm> ('a'->1|'b'->2|'a'->3)>-`
					`==>1`

,-poly Collections

The **,-polys** capture the additive abelian group of the mm-ADT stream ring.

abstract magma	stream magma	free poly
$(\texttt{obj}, +, \mathbf{0})$	$(\texttt{obj}, [\texttt{branch}], \{\mathbf{0}\})$	$(\texttt{obj}, (\texttt{,}), \{\mathbf{0}\})$

A general `,-poly` has `obj` as its carrier set, `,` as its the commutative binary $+$-operator, and `{0}` (`_{0}`) as its identity element. With `obj` quantification, should two `,-poly` terms have equal *objects*, they can be merged according to the mm-ADT `[branch]` operator equation:

$$[\texttt{branch}, a_q, b_r] = \begin{cases} a_{q+r} & \text{if } a == b, \\ [\texttt{branch}, a_q, b_r] & \text{otherwise,} \end{cases}$$

where $+$ is the quantifier ring's additive operator and

$$[a_q, \{\mathbf{0}\}] = a_q.$$

The commutative aspect of the `,-poly` does not enforce an order upon its elements which yields set-like semantics. However, given quantifier "weighting," `,-lst` collections realize multiset semantics (also known as bags or weighted sets) and `,-rec` collections realize multimap semantics (associative arrays with multiple values for a single key).

A few self-explanatory `,-poly` examples are provided below.

,-lst	,-rec

`mmlang> (1,{0},1,2,3)`
`==>(1{2},2,3)`
`mmlang> ('a'{7},'b','b'{0},'c','a'{2})`
`==>('b','c','a'{9})`
`mmlang> ('a',1.0,1,true)`
`==>('a',1.0,1,true)`
	`mmlang> (1->2,{0}->2,1->3,1->2,3->1)`
	`==>(1->2{2},3->1)`
	`mmlang> ('a'{7}->2,'a'->2{3},'b'->2)`
	`==>('a'{7}->2,'a'->2{3},'b'->2)`
	`mmlang> ('a'->true,1.0->6,1->{0},'a'->'a')`
	`==>('a'->[true,'a'],1.0->6)`

;-poly Collections

The **;-polys** capture the non-commutative, multiplicative monoid of the mm-ADT ring.

abstract magma	stream magma	free poly
$(\texttt{obj}, *, \mathbf{1})$	$(\texttt{obj}, [\texttt{juxt}], [\texttt{noop}])$	$(\texttt{obj}, (\texttt{;}), \{\mathbf{1}\})$

The `;-poly` carrier set is `obj`, the multiplicative operator is `;`, the multiplicative identity is `{1}`, and due to the larger ring in which this magma is embedded, `{0}` is the annihilator. Due to non-commutativity, the `;` delimited elements form an ordered sequence. In `lst`, the consequence is a collection data structure with list-semantics. In `rec`, a record with ordered fields is realized. Both support duplicates so the `rec` form is less like an associative array and more like a *list of pairs/fields*.

The `[juxt]` operator is mm-ADT's multiplicative monoid operator and is only applied in `;-poly` for the universal elements $\mathbf{1}$ and $\mathbf{0}$. Given the equation

$$a_q[\texttt{juxt}, b_r] = \begin{cases} b_{q*r} & \text{if } b \text{ is a value,} \\ b(a)_{q*r} & \text{otherwise,} \end{cases}$$

`;-poly` only computes the free monoid identity

$$a_q[\texttt{juxt}, [\texttt{noop}]] = a_q.$$

Examples are presented below that contain both $\{\mathbf{0}\}$ and $\{\mathbf{1}\}$ elements.

;-lst	;-rec

`mmlang> (1;6;1;2;{1})`
`==>(1;6;1;2;_)`
`mmlang> ('a'{7};'b';'b'{0};'c';'a'{2})`
`==>('a'{7};'b';{0};'c';'a'{2})`
`mmlang> ('a';{1};1.0;1;true;{1})`
`==>('a';_;1.0;1;true;_)`
	`mmlang> (1->2;{1}->2;1->3;1->{1};3->1)`
	`==>(1->2;_->2;3->1)`
	`mmlang> ('a'{7}->2;'a'->2{3};'b'->2)`
	`==>('a'{7}->2;'a'->2{3};'b'->2)`
	`mmlang> ('a'->true;1.0->4;1->{0};{1}->'a')`
	`==>({0}->{0})`

|-poly Collections

The **|-polys** captures endomorphic, left-distributive, multiplicative, compositions over the near-ring subgroup of mm-ADT's additive abelian group.

abstract magma	stream magma	free poly
$(\texttt{obj}, \oplus_1, \mathbf{0}/\mathbf{1})$	$(\texttt{obj}, [\texttt{barrier}, [\texttt{head}]], \{\mathbf{1}\}, \{\mathbf{0}\})$	$(\texttt{obj}, (\texttt{\|}), \{\mathbf{1}\}, \{\mathbf{0}\})$

The `[barrier]` n-ary operator's arguments are all the `objs` of the input stream. This yields a blocking synchronization point necessary for reduce/fold-based computations. The operator's 1 subscript denotes a particular augmentation to the higher-order $\oplus$ operator, where $\oplus_1$ returns the first non-zero `obj` argument—i.e., the head of the stream (a lazy computation).

$$[a_q, \ldots, b_r]\texttt{[barrier,[head]]} = \begin{cases} a_q & \text{if } q > 0, \\ \cdots & \\ b_r & \text{if } r > 0, \\ \{\mathbf{0}\} & \text{otherwise,} \end{cases}$$

`|-poly` yields singleton `lsts` and `recs`. The purpose of this seemingly odd behavior is more salient in `|-polys` flow controls (presented in the next section). A collection of self-explanatory examples are provided below.

|-lst	|-rec

`mmlang> (1|6|1|2|{1})`
`==>(1|6|2)`
`mmlang> ('a'{0}|'b'{0}|'c'|'a'{2})`
`==>('c'|'a'{2})`
`mmlang> ('a'{?}|{1}|1.0|true)`
`==>('a'{?}|_)`
	`mmlang> ({1}->2|6->2|1->3|1->{1}|3->1)`
	`==>(_->2)`
	`mmlang> ('a'{0}->2|'b'->2{3}|'c'->2)`
	`==>('b'->2{3})`
	`mmlang> ('a'->{0}|{0}->4|1->true|2->'a')`
	`==>(1->true)`

When each `poly` contains 0 or 1 element, the respective algebras behave equivalently. It is only at 2+ terms that the `poly` algebras become discernible and instructions such as `[eq]` consider the `poly` element separator in their calculation.

Poly Controls

The mm-ADT ring's additive abelian group operator is accessible via the `[branch]` instruction. The `[branch]` instruction's argument is a `poly`. Each term of the `poly` argument is an operand of the ring's $+$-operator. In this way, each of the 6 `poly` forms represents a particular control structure. Due to the prevalent use of `[branch]`, `mmlang` offers the sugar'd encoding of `[ ]`, where both the instruction opcode and the `poly` parentheses are not written. For example, `[branch,(+1,+2,+3)]` is equivalent to `[+1,+2,+3]`.

lst	control	mmlang example	rec	control	mmlang example

`,`	union cascade	`mm> 6 => int[+1,+3,+1]`
		`==>9`
		`==>7{2}`
			`,`	conditional cascade	`mm> 6 => int[is>10 -> +1,`
					`is>5 -> +2,`
					`int -> +3]`
					`==>8`
					`==>9`

`;`	fluent chaining	`mm> 6 => int[+1;+2;+3]`
		`==>12`
			`;`	conditional chaining	`mm> 6 => int[is>0 -> +1;`
					`is>5 -> +2;`
					`int -> +3]`
					`==>12`

`|`	coalesce	`mm> 6 => int[+1[is>10]|`
		`+2[is>5] |`
		`+3 ]`
		`==>8`
			`|`	switch	`mm> 6 => int[is>10 -> +1|`
					`is>5 -> +2|`
					`_ -> +3]`
					`==>8`

In the examples to follow, the elements of the `polys` are labeled by first letter of their algebraic structure: g is **abelian group**, m is **monoid**, and r is **near-ring**. For the `rec` examples, the key elements are labeled with k. The `mmlang` examples use Unicode subscript characters and `[plus]` to compose x with the `objs` along the respective data control path. These choices were purely aesthetic, driven by the desire to better align the examples with their respective illustrative diagrams and `mm` notation equations.

,-poly Controls

The **,-polys** capture the additive abelian group of the mm-ADT stream ring. The associativity and commutativity of the group operator means that the order in which the terms are evaluated (associativity) and results aggregated (commutativity) does not change the semantics of the computation. More specifically to the notion of control, it means that the irreducible terms in a `,-poly` are not sequentially dependent on one another. This independence enables evaluation isolation and thus, promotes parallelism. The `,-poly` algebra realizes cascading union in `lst` and conditional cascading in `rec`.

Table 2. ,-poly Branch

,-lst (union cascade)	,-rec (conditional cascade)
$x \Rightarrow [g_0, g_1, \ldots, g_n] = \coprod_{i=0}^{n} x \Rightarrow g_i$	$x \Rightarrow [(k_0 \to g_0), \ldots, (k_n \to g_n)] = \coprod_{i=0}^{n} \begin{cases} x \Rightarrow g_i & \text{if } (x \Rightarrow k_i) \neq \mathbf{0}, \\ \mathbf{0} & \text{otherwise.} \end{cases}$

`mm> 'x' => [+'g₀',+'g₁',+'g₂']`
`==>'xg₀'`
`==>'xg₁'`
`==>'xg₂'`
`mm> [['x']+'g₀',['x']+'g₁',`
`['x']+'g₂']`
`==>'xg₀'`
`==>'xg₁'`
`==>'xg₂'`
	`mm> 'x' => [+'k₀'->+'g₀',+'k₁'->+'g₁',+'k₂'->+'g₂']`
	`==>'xg₀'`
	`==>'xg₁'`
	`==>'xg₂'`

Branching can be understood as a two stage process which is captured by the `[split]`/`[merge]` instruction pairing. In practice, `[split]`/`[merge]` are useful in isolating a `poly` computation (`[split]`) to then later introduce the results (`[merge]`) into the outer stream. This is demonstrated below for `,-lst` and `,-rec` where `-<` is `mmlang` sugar for `[split]` and `>-` is `mmlang` sugar for `[merge]`.

Table 3. ,-poly Split/Merge

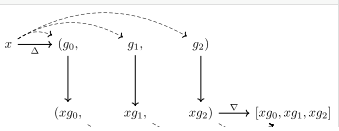
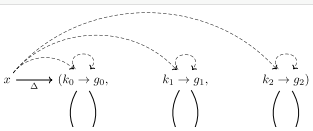
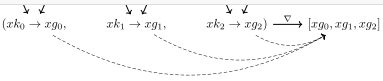
,-lst (union cascade)	,-rec (conditional cascade)
	

Table 3. ,-poly Split/Merge

,-lst (union cascade)	,-rec (conditional cascade)
	

`mm> 'x'-<(+'g₀',+'g₁',+'g₂')`
`==>('xg₀','xg₁','xg₂')`
`mm> 'x'-<(+'g₀',+'g₁',+'g₂')>-`
`==>'xg₀'`
`==>'xg₁'`
`==>'xg₂'`
	`mm> 'x'-<(+'k₀'->+'g₀',+'k₁'->+'g₁',+'k₂'->+'g₂')`
	`==>('xk₀'->'xg₀','xk₁'->'xg₁','xk₂'->'xg₂')`
	`mm> 'x'-<(+'k₀'->+'g₀',+'k₁'->+'g₁',+'k₂'->+'g₂')>-`
	`==>'xg₀'`
	`==>'xg₁'`
	`==>'xg₂'`

Note that in all subsequent `[branch,poly]` equations to follow, $x \in$ `obj` is an incoming `obj` to the respective `[branch]` instruction.

;-poly Controls

Monoids

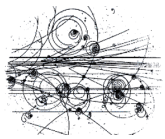
A monoid is a structure of the form $(A,\ \cdot\ ,\ \mathbf{1})$, where A is the carrier set closed under the associative binary operator $\cdot : A \times A \to A$ with $\mathbf{1} \in A$ being the identity such that for every $a, b, c \in A$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ and $a \cdot \mathbf{1} = \mathbf{1} \cdot a = a$.

The **;-polys** capture the multiplicative monoid of the mm-ADT ring. The result of each term is the input to the next term in the sequence. In `lst`, method chaining is realized and in `rec` conditional chaining.

Table 4. ;-poly Branch

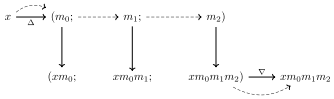
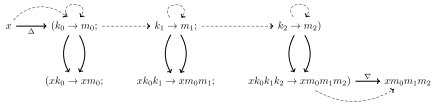
;-lst (fluent chaining)	;-rec (conditional chaining)
$x \Rightarrow [m_0; m_1; \ldots; m_n] \ = \ x \Rightarrow \prod_{i=0}^{n} m_i$	$x \Rightarrow [(k_0 \to m_0), \ldots, (k_n \to m_n)] \ = \ x \Rightarrow \prod_{i=0}^{n} \begin{cases} m_i & \text{if } (x \Rightarrow k_i) \neq \mathbf{0}, \\ \mathbf{0} & \text{otherwise.} \end{cases}$

`mm> 'x' => [+'m₀';+'m₁';+'m₂']`
`==>'xm₀m₁m₂'`
`mm> 'x' => +'m₀'+'m₁'+'m₂'`
`==>'xm₀m₁m₂'`
`mm> 'x' => +'m₀m₁m₂'`
`==>'xm₀m₁m₂'`
	`mm> 'x' => [+'k₀'->+'m₀';+'k₁'->+'m₁';+'k₂'->+'m₂']`
	`==>'xm₀m₁m₂'`



The `;-lst` equation above realizes a structure and process joyfully named the "bubble chamber". In experimental higher-energy physics, a bubble chamber is small room filled with high pressure vapor. Particles are shot into the chamber and the trace they leave (called their *varpor trail*) provides physicists empirical data regarding the nature of the particle under study — e.g., its mass, velocity, spin, and, when capturing decay, the sub-atomic particles that compose it. As demonstrated below, x is the *particle* and `-<` *shoots* x into the `;-lst` *bubble chamber*, where each term in the `;-lst` acts as a *vapor droplet*.

Table 5. ;-poly Split/Merge

;-lst (fluent chaining)	;-rec (conditional chaining)
	

`mm> 'x'-<(+'m₀';+'m₁';+'m₂')`
`==>('xm₀';'xm₀m₁';'xm₀m₁m₂')`
`mm> 'x'-<(+'m₀';+'m₁';+'m₂')>-`
`==>'xm₀m₁m₂'`
	`mm> 'x'-<(+'k₀'->+'m₀';+'k₁'->+'m₁';+'k₂'->+'m₂')`
	`==>('xk₀'->'xm₀';'xm₀k₁'->'xm₀m₁';'xm₀m₁k₂'->'xm₀m₁m₂')`
	`mm> 'x'-<(+'k₀'->+'m₀';+'k₁'->+'m₁';+'k₂'->+'m₂')>-`
	`==>'xm₀m₁m₂'`

|-poly Controls

The **|-polys** capture mm-ADT's *barrier* near-ring, where the first non-$\mathbf{0}$ ("non-null") element is the output of the branch. As a control structure, `|-poly` is a sequential branch that can be understood programmatically as a short-circuit fold. In `lst`, non-null coalescing is realized and in `rec` a switch statement is realized.

Table 6. |-poly Branch

|-lst (coalesce)	|-rec (switch)
$x \Rightarrow [r_0, r_1, \ldots, r_n] = \begin{cases} x \Rightarrow r_0 & \text{if } (x \Rightarrow r_0) \neq \mathbf{0}, \\ x \Rightarrow r_1 & \text{if } (x \Rightarrow r_1) \neq \mathbf{0}, \\ \ldots & \\ x \Rightarrow r_n & \text{if } (x \Rightarrow r_n) \neq \mathbf{0}, \\ \mathbf{0} & \text{otherwise.} \end{cases}$	$x \Rightarrow [(k_0 \to r_0), \ldots, (k_n \to r_n)] = \begin{cases} x \Rightarrow r_0 & \text{if } (x \Rightarrow k_0) \neq \mathbf{0}, \\ x \Rightarrow r_1 & \text{if } (x \Rightarrow k_1) \neq \mathbf{0}, \\ \ldots & \\ x \Rightarrow r_n & \text{if } (x \Rightarrow k_n) \neq \mathbf{0}, \\ \mathbf{0} & \text{otherwise.} \end{cases}$

`mm> 'x' => [+{0}'r₀'|+'r₁'|+'r₂']`
`==>'xr₁'`
`mm> [['x']+{0}'r₀'|['x']+'r₁']`
`==>'xr₁'`
	`mm> 'x' => ['k₀'{0}->+'r₀'|+'k₁'->+'r₁'|+'k₂'->+'r₂']`
	`==>'xr₁'`

Table 7. |-poly Split/Merge

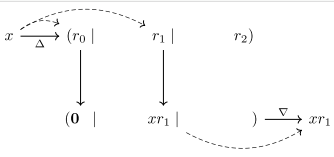
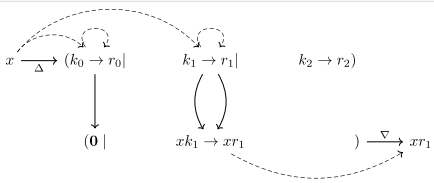
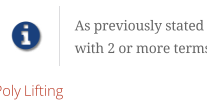
|-lst (coalesce)	|-rec (switch)
	

Table 7. |-poly Split/Merge

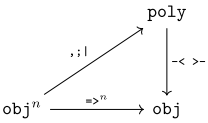
|-lst (coalesce)	|-rec (switch)

`mm> 'x'-<(+{0}'r₀'|+'r₁'|+'r₂')`
`==>('xr₁')`
`mm> 'x'-<(+{0}'r₀'|+'r₁'|+'r₂')>-`
`==>'xr₁'`
	`mm> 'x'-<('k₀'{0}->+'r₀'|+'k₁'->+'r₁'|+'k₂'->+'r₂')`
	`==>('xk₁'->'xr₁')`
	`mm> 'x'-<('k₀'{0}->+'r₀'|+'k₁'->+'r₁'|+'k₂'->+'r₂')>-`
	`==>'xr₁'`



As previously stated for collection **polys** , control **poly** semantics are only discernible amongst **polys** with 2 or more terms.

Poly Lifting



A consequence of the dual use of **poly** as both a data structure and a control structure is that **poly** supports a lifted encoding of mm-ADT itself. Each **poly** form captures a particular magma of the underlying mm-ADT stream ring algebra. As a collection, **poly** provides a programmatic way of writing mm-ADT programs (types) and as flow control, these **poly** encoded mm-ADT programs can be executed. The complete algebraic specification of **poly** lifting via an **obj** -module of the mm-ADT ring will be presented in a latter section. For now, the following **mmlang** examples demonstrate **poly** lifting in support of mm-ADT metaprogramming.

The mm-ADT type below contains both monoidal (serial composition) and group (parallel branching) components whose construction is captured by the bottom morphism of the diagram above. Note that the **[explain]** instruction is appended for educational purposes only—so as to detail the ⇒ compositions.

$$\mathtt{obj}^n \xrightarrow{\mathtt{=>}^n} \mathtt{obj}$$

```
mmlang> int{3}[mult,10][is>20 -> [+70,+170,+270],
                        is>10 ->   [*10,*20,*30]][plus,100][explain]
==>'
int{0,18}<=int{3}[mult,10][int{0,3}<=int{3}[is,bool{3}<=int{3}[gt,20]]->int{9}<=int{3}[int{3}
[plus,70],int{3}[plus,170],int{3}[plus,270]],int{0,3}<=int{3}[is,bool{3}<=int{3}[gt,10]]->int{9}<=int{3}
[int{3}[mult,10],int{3}[mult,20],int{3}[mult,30]]][plus,100]

inst                                          domain            range       state
-------------------------------------------------------------------------------------
[mult,10]                                     int{3}       =>   int{3}
[int{0,3}<=int{3}[is,bool{3}<=int{3}[gt,...   int{3}       =>   int{0,18}
 [is,bool{3}<=int{3}[gt,20]]                   int{3}       =>    int{0,3}
  [gt,20]                                      int{3}       =>     bool{3}
 ->[int{3}[plus,70],int{3}[plus,170],int{3}...   int{3}      =>    int{9}
  [plus,70]                                    int{3}       =>     int{3}
  [plus,170]                                   int{3}       =>     int{3}
  [plus,270]                                   int{3}       =>     int{3}
 [is,bool{3}<=int{3}[gt,10]]                  int{3}       =>    int{0,3}
  [gt,10]                                      int{3}       =>     bool{3}
 ->[int{3}[mult,10],int{3}[mult,20],int{3}[...   int{3}      =>    int{9}
  [mult,10]                                    int{3}       =>     int{3}
  [mult,20]                                    int{3}       =>     int{3}
  [mult,30]                                    int{3}       =>     int{3}
[plus,100]                                    int{0,18}    =>   int{0,18}
'
```

The above type can be expressed in a pure **poly** form, where **;** is serial composition and **,** is parallel branching. This construction is captured by the slanted morphism in the diagram above.

$$\mathtt{obj}^n \xrightarrow{\mathtt{,;|}} \mathtt{poly}$$

```
mmlang> (int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270)>-)>-,
                            -<([is>10];-<(*10,*20,*30  )>-)>-);[plus,100])
==>(int{3};_[mult,10];_[split,(_[split,(_[is,_[gt,20]];_[split,(_[plus,70],_[plus,170],_[plus,270])]
[merge])][merge],_[split,(_[is,_[gt,10]];_[split,(_[mult,10],_[mult,20],_[mult,30])][merge])][merge])]
[merge];_[plus,100])
```

The **[split]** instruction (sugar'd **-<**) renders **poly** a ring module. Incoming **objs** are scalars to a **poly** vector according to the equations

$$\begin{aligned} x \prec (v_0, v_1, \ldots, v_n) &= (xv_0, xv_1, \ldots, xv_n) \\ x \prec (v_0; v_1; \ldots; v_n) &= (xv_0; v_1; \ldots; v_n) \\ x \prec (v_0 | v_1 | \ldots | v_n) &= (xv_i), \end{aligned}$$

where $x \prec$ **poly** is the instruction **x => [split,poly]** . The **[merge]** instruction evaluates the **poly** according to the algebra denoted by its term separator (**,** , **;** , or **|**). This has the effect of "draining" the **poly** of it's internal **objs** such that

$$\begin{aligned} (xv_0, xv_1, \ldots, xv_n) \succ &= \prod_{i=0}^{n} x \Rightarrow v_i \\ (xv_0; v_1; \ldots; v_n) \succ &= x \Rightarrow \prod_{i=0}^{n} v_i \\ (xv_i) \succ &= xv_i : v_i \neq \mathbf{0}, \end{aligned}$$

where **poly** $\succ$ is the expression **poly => [merge]** .

Finally, both the original unlifted form and the **poly** lifted form of the type yield the same result at evaluation, where the final expression binds (**-<**) the values 1, 2, and 3 to the indeterminate terms, thus solving (**>-**) the polynomial equation.

$$\mathtt{poly} \xrightarrow{\mathtt{-<\ >-}} \mathtt{obj}$$

```
mmlang> [1,2,3] => int{3}[mult,10][is>20 -> [+70,+170,+270],
                                   is>10 ->   [*10,*20,*30]][plus,100]
==>500
==>200
==>300{2}
==>400{2}
==>700{2}
==>1000
mmlang> [1,2,3]-<(int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270)>-)>-,
                                      -<([is>10];-<(*10,*20,*30  )>-)>-)>-;[plus,100])>-
==>500
==>200
==>300{2}
==>400{2}
```

```
==>700{2}
==>1000
```

Given that `[split,poly:x][merge]` is equivalent to `[branch,poly:x]`, the `poly` type can be written more succinctly in a pure `[branch]` form as below.

```
mmlang> [1,2,3] => [int{3};[mult,10];[[[is>20];[+70,+170,+270]],
                                     [[is>10];[*10,*20,*30  ]]];[plus,100]]
==>500
==>200
==>300{2}
==>400{2}
==>700{2}
==>1000
```

Note that, when incident to each other, `[split]/[merge]` has the same equation as `[branch]`.

$$
\begin{aligned}
x \prec (v_0, v_1, \ldots, v_n) \succ &= x \Rightarrow \left[v_0, v_1, \ldots, v_n\right] &= \coprod_{i=0}^{n} x \Rightarrow v_i \\
x \prec (v_0; v_1; \ldots; v_n) \succ &= x \Rightarrow \left[v_0; v_1; \ldots; v_n\right] &= x \Rightarrow \prod_{i=0}^{n} v_i \\
x \prec (v_0 | v_1 | \ldots | v_n) \succ &= x \Rightarrow \left[v_0 | v_1 | \ldots | v_n\right] &= x v_i : v_i \neq \mathbf{0}
\end{aligned}
$$

The reason for using `-<( )>-` versus `[ ]` is that when `[split]` and `[merge]` are not juxtaposed, reflection is possible on the intermediate results of the internal `poly` computation. That is, when only a `[split]` is applied, a *half-branch* occurs and all the `poly` domain instructions can operate on the midway results. Intuitively, `[split]` transforms a control structure into a data structure and `[merge]` transforms a data structure into a control structure. At this intermediate point when the computation is a data structure, the computation can be manipulated programmatically. That is the power of a lifted representation.

```
mmlang> [1,2,3]-<(int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270))))
==>(1;10;(({0};{0})))
==>(2;20;(({0};{0})))
==>(3;30;((30;(100,200,300))))
mmlang> [1,2,3]-<(int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270)),
                                     -<([is>10];-<(*10,*20,*30  ))))
==>(1;10;(({0};{0}),({0};{0})))
==>(2;20;(({0};{0}),(20;(200,400,600))))
==>(3;30;((30;(100,200,300)),(30;(300,600,900))))
mmlang> [1,2,3]-<(int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270)>-)>-),
                                     -<([is>10];-<(*10,*20,*30  )>-)>-)>-;[plus,100])
==>(1;10;{0};{0})
==>(2;20;[200,400,600];[300,500,700])
==>(3;30;[100,200,300{2},600,900];[200,300,400{2},700,1000])
mmlang> [1,2,3]-<(int{3};[mult,10];-<(-<([is>20];-<(+70,+170,+270)>-)>-),
                                     -<([is>10];-<(*10,*20,*30  )>-)>-)>-;[plus,100])>-
==>500
==>200
==>300{2}
==>400{2}
==>700{2}
==>1000
```

In summary, mm-ADT can be embedded in `poly` itself. The formal proof of this fact demonstrates that the mm-ADT instruction set architecture, the two ring operators ($+$ and $\cdot$), and the *reduce* near-ring operator ($\oplus$) are sufficiently expressive to yield a Turing Complete computing machine.

The Obj Graph

An mm-ADT program is a **type**. The `mmlang` parser converts a textual representation of a type into a type `obj`. A type is inductively defined and is encoded as a path within the larger **type graph**. The type's path is a graphical encoding specifying a data flow pipeline that when evaluated, constructs elements of the type (i.e. computed resultant values). These values also have a graphical encoding paths in the **value graph**. Together, the type graph and the value graph form the quiver known as the **obj graph**.

Every aspect of an mm-ADT computation from composition, to compilation, and ultimately to evaluation is materialized in the `obj` graph. The following itemizations summarizes the various roles that the `obj` graph throughout a computation.

- **Composition**: The construction of a type via the point-free style of `mmlang` is a the lexical correlate of walking the `obj` graph from a source vertex (domain ctype) across a series of instruction-labeled edges (`inst`) to ultimately arrive at a target vertex (range ctype). The path, a free object, contains both the type's signature and definition.
- **Compilation**: A path in the type graph can be prefixed with another ctype (e.g. placing `int` before `_`). In doing so, the path's domain has been alterered and the path is recomputed to potentially yield a variant of the original path (e.g. a type inferenced path).
- **Rewrite**: Subpaths of a path in the type graph can be specified as being semantically equivalent to another path in the type graph via `poly` lifted rewriting `(y)<=(x)`. Subsequent compilations and evaluations of the path may yield path variants.
- **Optimization**: Every instruction in `inst` has an associated cost dependent on the underlying storage and processor. Rewrites create a superposition of programs. Given that the `obj` graph commutes, a weighted shortest path calculation from a domain vertex to a range vertex is an example of a simple technique for choosing an efficient execution plan.
- **Variables**: Variable bindings are encoded in instructions. When the current instruction being evaluated requires historic state information, the `obj` $^{\text{op}}$ graph (with edges reversed) is searched in order to locate the vertex incident to a variable `inst`.
- **Evaluation**: Program evaluation binds the type graph to the value graph. When a type path is prefixed with a value `obj`, the instructions along the path operate on the value, where the path's target vertex is the result of the computation.

This section will discuss the particulars of the aforementioned uses of the `obj` graph.

State

Let $(M, \cdot, e)$ be a monoid, where $e \in M$ is the identity element and there exists an element $e' \in M$ that also acts as an identity such that for every $x \in M, x \cdot e = x$ and $x \cdot e' = x$, then because $e \cdot e' = e$ and $e \cdot e' = e'$, it is the case

that $e = e \cdot e' = e'$ and $e = e'$. Thus, every monoid has a **single unique identity**. However, in a free monoid, where element composition history is preserved, it is possible to record e and e' as distinctly *labeled* elements even though their role in the non-free monoid's binary composition are the same — namely, that they both act as identities.

It is through **multiple distinct identities** in `inst` that mm-ADT supports the programming idioms in the associated table. The general approach is *state is stored along the path of the* `obj`.

```
mmlang> 6 => int[plus,[mult,2]][path]
==>(6;[plus,12];18)<=6[plus,12][path]
mmlang> 8 => int[plus,[mult,2]][path]
==>(8;[plus,16];24)<=8[plus,16][path]
```

idiom	inst	description
variables	`[to]`	`obj` references
type definitions	`[define]`	`type` mappings
models	`[model]`	domain of discourse
reversible computing	`[path]`	computing history

Every `obj` exists as a distinct vertex in the `obj` graph. If $b \in$ `obj` has an incoming edge labeled $i \in$ `inst`, then when applied to the outgoing adjacent vertex a, b is computed. Thus, the edge $a \rightarrow_i b$ records the instruction and incoming `obj` (a) that yielded the `obj` at the head of the edge (b). Since types are defined **inductively** and their respective values generated **deductively** via instruction evaluation along the type's **path**, the path contains all the information necessary to effect state-based computing. The path of an `obj` is accessed via the `[path]` instruction. The output of `[path]` is a `;-lst` — i.e., an element of the `inst` syntactic monoid. This path `lst` is also a product and as such, can be introspected via it's projection morphisms (e.g., via `[get]`).

```
mmlang> 8 => int[plus,1][mult,2][lt,63]                                    (1)
==>true
mmlang> 8 => int[plus,1][mult,2][lt,63][path]                              (2)
==>(8;[plus,1];9;[mult,2];18;[lt,63];true)<=8[plus,1][mult,2][lt,63][path]
mmlang> 8 => int[plus,1][mult,2][lt,63][path][get,5][get,0]                (3)
==>63
```

(1) The evaluation of an `bool<=int` type via 8.

(2) The `obj` graph path from 8 to `[lt,63]`.

(3) A projection of the instruction `[lt,63]` from the path and then the first argument of the `inst`.

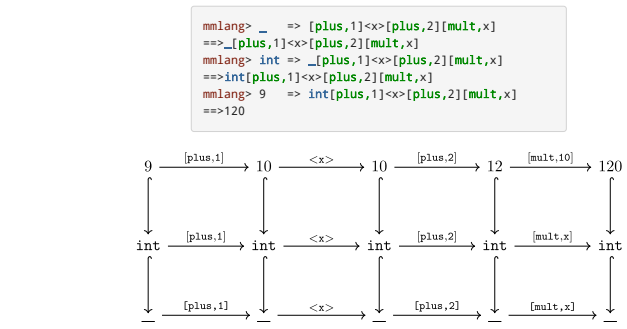
mm-ADT's **multiple identity instructions** simply compute the identity function $f(x) \mapsto x$, but as edge labels in the `obj` graph, they store state information that can be later accessed via trace-based path analysis (i.e. via `[path]`). In effect, the execution context is transformed from a memory-less finite state automata to a register-based Turing machine.

Variables

The `[to]` instruction's type definition is `a<=a[to,_]`. The argument to `[to]` is a **named anonymous type**. For every incoming $a \in$ `obj`, there is an outgoing a whose path has been extended with the `[to]` instruction. An example is provided below.

```
_[plus,1][to,x][plus,2][mult,x]
```

Suppose `int` is applied to the above anonymous type. This triggers a cascade of events whereby `[plus,1]` maps `int` to `int[plus,1]`, then `[to,x]` maps `int[plus,1]` to `int[plus,1][to,x]`, and so forth. The resultant compiled `int`-type can then be evaluated by an `int` value such as 9. In the commuting diagram below, the top instruction sequence forms a value graph (**evaluation**), the middle sequence a type graph (**compilation**), and the bottom, an *un*typed graph (**composition**). The union of these graphs via the inclusion morphism (`[type]`) is the complete `obj` graph of the computation.



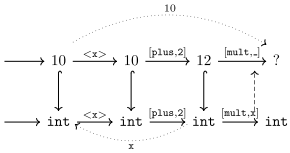
> In `mmlang`, the `[to]` instruction's sugar is `< >`. It is the only instruction whose sugar is printed as opposed to its `[ ]` form.

```
mmlang> _   => [plus,1]<x>[plus,2][mult,x]
==>_[plus,1]<x>[plus,2][mult,x]
mmlang> int => _[plus,1]<x>[plus,2][mult,x]
==>int[plus,1]<x>[plus,2][mult,x]
mmlang> 9   => int[plus,1]<x>[plus,2][mult,x]
==>120
```

The primary idea concerning variable state is that when `[mult,x]` is reached by the `int` value 12 via instruction application, the anonymous type `x` must be **resolved** before `[mult]` can evaluate. To do so, the instruction `[to,x]` is searched for in the path history of 12. When that instruction is found, the range (or domain as it's an identity) replaces `x` and `[mult,10]` is evaluated and the edge

$$12 \rightarrow_{[\texttt{mult},10]} 120$$

extends the value graph. The intuition for this process is illustrated on the right.




```
mmlang> 9 => int[plus,1]<x>[plus,2][mult,x][path]                          (1)
==>(9;[plus,1];10;<x>;10;[plus,2];12;[mult,10];120)<=9[plus,1]<x>[plus,2][mult,10][path]
mmlang> int[plus,1]<x>[plus,2][mult,x][explain]                            (2)
==>'
int[plus,1]<x>[plus,2][mult,x]

inst        domain      range state
-------------------------------------
[plus,1]    int     =>  int
[plus,2]    int     =>  int     x->int
[mult,x]    int     =>  int     x->int
'
```

(1) The `[path]` instruction provides the path of the current `obj` as a `;-lst`.

(2) The `[explain]` instruction details the scope of state variables.

The variable's scope starts at `[to]` and ends when there is no longer a path to `[to]`. If an `inst` argument is a type (e.g. `[mult,[plus,x]]`), then the **child type** (`[plus,x]`) path extends the **parent type** (`[mult]`) path. As such, the child type has access to the variables declared in the parent composition up to the `inst` containing the child type (`[mult]`). Finally, if `[to,x]` is evaluated and later along that path `[to,x]` is evaluated again, all subsequent types will resolve `x` at the latter `[to,x]` instruction. That is, the graph search halts at the first encounter of `[to,x]` — the shortest path to a declaration.

```
mmlang> 2 => int<x>[plus,<y>][plus,y]                                          ❶
language error: 4 does not contain the label 'y'
mmlang> 2 => int<x>[plus,[plus,x]<x>[plus,x]][plus,x]                          ❷
==>12
mmlang> 2 => int<x>[plus,[plus,x]<x>[plus,x]][plus,x][path]                    ❸
==>(2;<x>;2;[plus,8];10;[plus,2];12)<=2<x>[plus,8][plus,2][path]
mmlang> int<x>[plus,int<y>[plus,int<z>[plus,x][plus,y][plus,z]][plus,y]][plus,x][explain]   ❹
==>'
int<x>[plus,int<y>[plus,int<z>[plus,x][plus,y][plus,z]][plus,y]][plus,x]

inst                                          domain      range state
-------------------------------------------------------------------------
[plus,int<y>[plus,int<z>[plus,x][plus,y]...    int    =>   int    x->int
 [plus,int<z>[plus,x][plus,y][plus,z]]          int   =>    int   x->int y->int
  [plus,x]                                      int   =>    int   x->int y->int z->int
  [plus,y]                                      int   =>    int   x->int y->int z->int
  [plus,z]                                      int   =>    int   x->int y->int z->int
 [plus,y]                                      int   =>    int   x->int y->int
[plus,x]                                      int    =>   int    x->int
'
```

❶ The variable `y` is declared in a branch nested within the retrieving branch.

❷ The variable `x` is redefined in the nested branch and recovers its original value when the nested branch completes.

❸ The value path of the previous evaluation highlighting that the final `[plus,x]` resolved to `[plus,2]`.

❹ A multi-nested expression demonstrating the creation and destruction of variable scope.

Definitions

A **type definition** takes one of the two familiar forms

$$b \Leftarrow a$$

or

$$b : a$$

where, for the first, b is *generated* by a and for the second, b is *structured* as a and, when considering no extending instructions to the $b \Leftarrow a$ form, $b \Leftarrow a \cong b : a$ such that a is *named* b. For most of the documentation, the examples have been presented solely from within the `mm` model-ADT where there are 6 types: `bool`, `int`, `real`, `str`, `lst`, and `rec` along with their respective instructions. It is possible to extend `mm` with new types that are ultimately *grounded* (Cayley rooted) in the `mm` model-ADT types. This is the purpose of the `[define]` instruction which will now be explained by way of example.

The natural numbers ($\mathbb{N}$) are a refinement of the set of integers ($\mathbb{Z}$), where $\mathbb{N} \subset \mathbb{Z}$. In set builder notation, specifying the set of integers and a predicate to limit the set to only those integers greater than 0 is denoted

$$\mathbb{N} = \{n \in \mathbb{Z} \mid n > 0\}.$$

In mm-ADT, `int` is a `nat` ($\mathbb{N}$) if there is a **path** through the type graph from the `int` to `nat`. These paths are **type definitions**. In the example below, `[define]` creates a path from at `int` to `nat` via the instruction `[is>0]`.

```
[define,nat<=int[is>0]]
```

A `nat` is any `int` that arrives at `nat` via `nat<=int[is>0]`. Given this definition (and this definition only), `nat` is a *refinement* of `int` because only 50% of `int`s successfully reach `nat`. However, there may be other paths to `nat` from other types and as such, type refinement is a relative concept in mm-ADT. In isolation, `nat` is only a character label (called a **name**) attached to a vertex in the `obj` graph. There is no other structure to a isolated type. The nature of a type is completely determined by the paths incoming and outgoing from it. In this graph-based interpretation of types, a type can be the source or target of any number of paths and it is through navigating these paths that values at a type are morphed into values at other types, where mm-ADT instructions (`inst`) specify, step-by-step, the way in which the morphing process is to be carried out.

```
mmlang> 36 => int[define,nat<=int[is>0]]                    ❶
==>36
mmlang> 36 => nat<=int[define,nat<=int[is>0]]               ❷
==>nat:36
mmlang> 36 => nat<=int[define,nat<=int[is>0]][mult,-1]      ❸
language error: -36 is not a nat
```

❶ A `nat` is defined, but never applied. Thus, logically, this is equivalent to `36 => int`.

❷ A type can be used anytime after its definition in the path. Thus, `nat` is a viable range type.

❸ If the `obj` is not a `nat`, then the larger `nat<=int` is invalid.

Deep Dive 3. mm-ADT Type Prefix

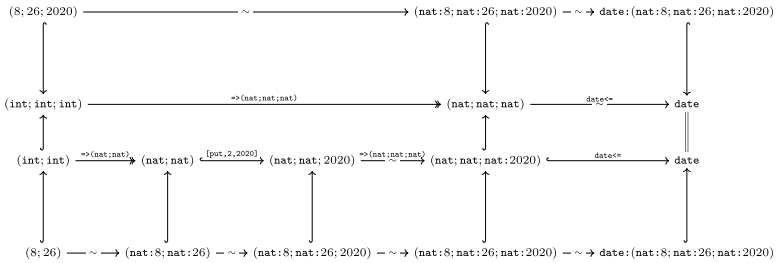
Prepending type definitions to every program reduces legibility and complicates program development. For this reason, mm-ADT provides a **type prefix**. All `mmlang` examples that start with `:` are defining *the* type prefix that will be used for all subsequent programs. The type prefix is a generalization of a library statement such as `import` or `module` found in other programming languages. The generalization is that a type prefix can be any type, not just those containing only `[define]`). The type prefix is prepended to the program type prior to compilation, where this operation is made sound by the free `inst` monoid.

```
mmlang> :[model,mm][define,nat<=int[is>0]]
==>_[define,nat<=int[is,bool<=int[gt,0]]]
mmlang> 10 => nat
==>nat:10
mmlang> -1 => nat
language error: -1 is not a nat
mmlang> 10 => nat[plus,5]
==>nat:15
mmlang> 10 => nat[plus,5][plus,-15]
language error: 0 is not a nat
```

The example below defines a `date` to be a `;-lst` with 2 or 3 `nats`. If the `;-lst` contains

↪	inclusion
↣	monomorphism
↠	epimorphism
∼	isomorphism
=	equality

only 2 terms, then a default value of 2020 is provided. This highlights an important aspect of mm-ADT's type system. Variables, types, and rewrites are all *graph search processes*. A defined type (path) with a desired **range** is searched for in the `obj` graph and returned if and only if the morphing `obj` matches the defined type's **domain**. Type definitions are simply other types that specify the means by which one type is translated into another type. To the left, the meaning of the arrows' graphical annotations are provided.




```
mmlang> :[model,mm][define,nat<=int[is>0],
                    date:(nat[is=<12];nat[is=<31];nat),
                    date<=(nat[is=<12];nat[is=<31])[put,2,2020]]
mmlang>
mmlang> (8;26;2020) => date
==>date:(nat:8;nat:26;nat:2020)
mmlang> (8;26)      => date
==>date:(nat:8;nat:26;nat:2020)
```

Defining types with `[define]` is useful for *in situ* definitions that are only require through the scope of the definition (typically within nested types). For reusing types across mm-ADT programs, mm-ADT offers *models* and the `[model]` instruction.

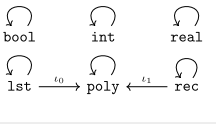
Models

Homotopy Type Theory

Homotopy type theory understands types as coexisting with other types in a topological space. A spatial embedding implies that a type can be reached from another type by moving through space. The way in which the space is navigated is via paths. A path is a continuous deformations of one type into another type.

Types can be organized into **model-ADTs** (simply called **models**). The 4 *mono* types (`bool` , `int` , `real` , `str`) and the 2 *poly* types (`lst` , `rec`) are defined in the `mm` model-ADT (the *mm* of *mm*-ADT). The instruction `[model,mm]` generates a `rec` from the `mmlang` file `mm.mm` . Using the same multiplicity of identities principle, the `rec` is accessible in the type's path definition via the `[model]` argument.

```
mm:('type' ->
  (bool ->  ( ), int ->  ( ), real -> ( ),
   str  ->  ( ), lst  -> ( ), rec ->  ( ),
   poly ->  (poly<=[lst|rec]),
))
```



The `rec` encoding of a model-ADT has the model's canonical types (**ctypes**) as keys and `lsts` of derived types (**dtypes**) as values. The encoding is a serialization of a graph where the ctypes are vertices and the incoming paths to a ctype vertex are the edges. Unfortunately, the `mm` model is too basic to demonstrate this point clearly. What `mm` does capture is the set nature of the base types in that there are vertices and no edges (save `poly` which is the coproduct of `lst` and `rec`).

In general, any model $\mathbf{m}$ is defined

$$\begin{aligned}\texttt{model}_{\mathbf{m}} &= \coprod_{i=0}^{|\mathbf{m}|} \texttt{ctype}_i + (\texttt{dtype}_i^0 + \texttt{dtype}_i^1 + \ldots + \texttt{dtype}_i^n) \\ &= \coprod_{i=0}^{|\mathbf{m}|} \texttt{ctype}_i + \coprod_{j=0}^{|\texttt{dtype}_i|} \texttt{dtype}_i^j.\end{aligned}$$

There are more ctypes than the 6 base types specified in `mm` . Typically, a ctype in one model is a dtype in another. If model $\mathbf{m}$ has $\texttt{ctypes}_{\mathbf{m}}$ derived from types in model $\mathbf{n}$, then $\texttt{dtypes}_{\mathbf{n}} \subseteq \texttt{ctypes}_{\mathbf{m}}$. However, `mm` is unique in that the `mm` types are *universally grounded* and

$$\begin{aligned}\mathbf{mm} &= \coprod_{i=0}^{6} \texttt{ctype}_i + \mathbf{0}_i \\ &= \coprod_{i=0}^{6} \texttt{ctype}_i \\ &= (\texttt{bool} + \texttt{int} + \texttt{real} + \texttt{str} + \texttt{lst} + \texttt{rec})\end{aligned}$$

That is, `mm` is the sum of 6 ctypes—the mm-ADT base types. Within `mm` , these ctypes are identity types. For example, in the `mm` model `rec` at the beginning of this section, the field `bool -> ( )` denotes $\texttt{bool} + \mathbf{0}$ or simply `bool`. The `bool` ctype is shorthand for `bool<=bool` , which, when considering the quantifier ring, is shorthand for `bool{1}<=bool{1}` . An instruction less type is a `noop` and thus, `bool` captures the reflexivity of identity:

$$\texttt{bool} \Leftarrow \texttt{bool} \equiv \texttt{bool} + \mathbf{0} \equiv \texttt{bool}.$$

Deep Dive 4. Model-ADT Subgraphs of the `obj` Graph

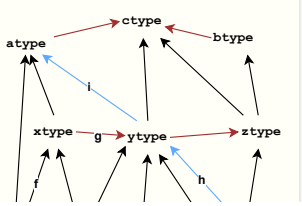
The associated illustration presents 3 models, their respective ctypes, and various dtypes between them. Every directed labeled binary edge in the diagram is a type of the form:

```
b<=a[inst{*}]
```

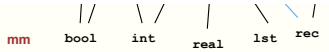
A type definition's instructions specify the specific, discrete

amodel

xmodel



computational steps (`inst`) necessary to transform `a` (domain) into `b` (range). A series of instructions are constructed with type induction (composition), destructed with type deduction (compilation or evaluation), and are captured as paths in the type subgraph of the `obj` graph. Thse paths are equivalent to the morphisms of the `obj` category diagram and the edges in the `obj` Cayley graph. The illustration highlights three sorts of types:



- `xtype<=int[f]` (**intra-model**): In `xmodel` , `xtype` is grounded at `int` in `mm` .
- `ytype<=xtype[g]` (**inter-model**): In `xmodel` , `ytype` can be reached via `xtype` .
- `atype<=rec[h][i]` (**trans-model**): In `amodel` , `atype` is grounded at `rec` in `mm` via `ytype` in `xmodel` .

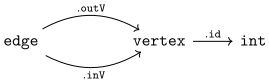
The `mm` model-ADT is too simple to be informative. The complexity of its types exist outside the virtual machine. In order to provide a comprehensive understanding of mm-ADT models, the following sections will build a **property graph** model-ADT (`pg`) in stages starting with `pg_1` , then `pg_2` , so forth before reaching the final complete encoding in `pg` .

Constructors

Property Graph Model 1

Graph theory acknowledges a variety of graph structures. One such structure is the **property graph**. The more descriptive, yet significantly longer name is the *directed, attributed, multi-relational binary graph*. The breadth of features will ultimately be captured in `pg` . The reduced `pg_1` model only defines a *directed binary graph*. In `pg_1` , a `vertex` can be derived from a `rec` with an `'id'->int` field. An edge can be derived from a `rec` with an outgoing/start `vertex` (`outV`) and an incoming/end `vertex` (`inV`). The associated diagram graphically captures the `pg_1` structure, where the `.` prefixes on the `inst` morphisms denote the `mmlang` sugar notation for `[get]` —e.g., `.outV` is sugar for `[get,'outV']` .

```
pg_1:('import' -> (mm -> ()),
'type' -> (
  vertex -> (vertex:('id'->int)),
  edge   -> ( edge:('outV'->vertex,'inV'->vertex))))
```



A type definition with no instructions serves as both a model constructor and canonical type (a ctype). As a canonical type, the path from source to target does nothing. An `obj` that matches the left-hand side is simply labeled with the name of the `obj` on right-hand side.

$$\texttt{(id} \to \texttt{int)} \xrightarrow{[\text{noop}]} \texttt{vertex.}$$

Three examples of constructing a `vertex` are presented below.

```
mmlang> :[model,pg_1]                                  ❶
==>_
mmlang> ('id'->1) => vertex                            ❷
==>vertex:('id'->1)
mmlang> ('id'->1,'age'->28) => vertex                  ❸
==>vertex:('id'->1,'age'->28)
mmlang> ('ID'->1) => vertex                            ❹
language error: ('ID'->1) is not a vertex
```

❶ The type prefix loads the `pg_1` model into the `obj` graph.

❷ A `vertex` from a `rec` with the requisite `'id'` field.

❸ Extraneous (non-ambiguous) in the `vertex` instance is mapped to the terminal **0**.

❹ Coercion to a `vertex` is not possible given as `'ID'` is not `'id'` .

Three `edge` construction examples are presented below.

```
mmlang> :[model,pg_1]
==>_
mmlang> ('outV'->vertex:('id'->1),'inV'->vertex:('id'->2)) => edge     ❶
==>edge:('outV'->vertex:('id'->1),'inV'->vertex:('id'->2))
mmlang> ('outV'->('id'->1),'inV'->('id'->2)) => edge                   ❷
==>edge:('outV'->vertex:('id'->1),'inV'->vertex:('id'->2))
mmlang> (vertex:('id'->1);vertex:('id'->2)) => edge                    ❸
language error: (vertex:('id'->1);vertex:('id'->2)) is not an edge
```

❶ An `edge` is the `rec` product of two `vertices` .

❷ If the components of the product can be coerced into vertices, they are automatically done so.

❸ A `lst` product is not the same as a `rec` product given that `recs` are products of key/value pairs.

Type Paths

Property Graph Model 2

In the previous `pg_1` model, a `vertex` (`edge`) was constructed using a `rec` with the requisite component structure. After validating the structural type of `rec` , the `rec` is labeled `vertex` (`edge`). There are situations in which the *source* `obj` has a significantly different absolute structure than the *target* `obj` . The ways in which an `obj` can be constructed are categorized in the table below where **inline** is for one time use, **define** for repeated use in a program, and **model** for reuse across programs.

inline

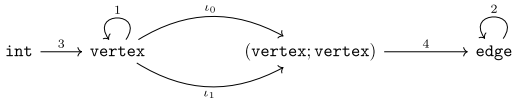
```
mmlang> :[model,pg_1]
==>_
mmlang> 1 => vertex<=int-<('id'->_)
==>vertex:('id'->1)
```

define

```
mmlang> :[model,pg_1][define,vertex<=int-<('id'->_)]
==>_[define,vertex<=int[split,('id'->int)]]
mmlang> 5 => vertex
==>vertex:('id'->5)
mmlang> (5;6) => (vertex;vertex)
==>(vertex:('id'->5);vertex:('id'->6))
```

model

```
pg_2:('import' -> (mm -> ()),
'type'   -> (
  vertex -> (vertex:('id'->int),                                  ❶
             vertex<=int-<('id'->_)),                             ❷
  edge   -> (edge:('outV'->vertex,'inV'->vertex),                 ❸
             edge<=(vertex;vertex)-<('outV'->.0,'inV'->.1))))     ❹
```



1. The `mm` specification of a canonical `vertex` (an `inst`-less ctype).
2. A path from an `int` to a `vertex`.
3. The `mm` specification of a canonical `edge` (an `inst`-less ctype).
4. A path from a 2-tuple `vertex ;-lst` to a `edge`.

An edge can be constructed in a number ways. The constructor below maps an `int` pair ($\mathbb{Z} \times \mathbb{Z}$) to an `edge` by propagating the pair into the edge product and then constructing vertices for the `outV` and `inV` fields. This structure has sufficient information for rendering the final `edge`. The mm-ADT VM simply names the `rec` pair elements `vertex` and the outer `rec edge` thus completing the transformation of an `(int;int)` to `edge` given `pg_1`.

```
mmlang> :[model,pg_2]
==>_
mmlang> (5;6) => (vertex;vertex) => edge                    1
==>edge:('outV'->vertex:('id'->5),'inV'->vertex:('id'->6))
mmlang> (5;6) => edge<=(vertex;vertex)
==>edge:('outV'->vertex:('id'->5),'inV'->vertex:('id'->6))
mmlang> (5;6) => edge                                       2
==>edge:('outV'->vertex:('id'->5),'inV'->vertex:('id'->6))
mmlang> 5 => int-<(vertex;vertex) => edge
3
==>edge:('outV'->vertex:('id'->5),'inV'->vertex:('id'->5))
mmlang> 5-<(vertex;vertex)=>edge
==>edge:('outV'->vertex:('id'->5),'inV'->vertex:('id'->5))
```

1. An `int` pair morphed into a `vertex` pair and then into an `edge`.
2. An `int` pair morphed into an `edge`.
3. An `int` split into `vertex` clone pairs and then morphed into a self-loop edge.

The final example above demonstrates the use of `;-lst` as both a coproduct and a product — i.e., a biproduct. The `(vertex;vertex)` pair is created via a split `-<` which serves as the coproduct injections ι_0 and ι_1. From this vertex coproduct, the `edge` definition projects out each component of via `.0` (`[get,0]`) and `.1` (`[get,1]`). Thus, the coproduct is also a product. For this reason, the Unicode character for pi (π) (the conventional symbol for product projection) serves as another `mmlang` sugar for `[get]`.

Deep Dive 5. model-ADT Application Programming Interfaces

The software development pattern espoused by mm-ADT is one in which software libraries (APIs) are large commuting diagrams constructed via domain/range concatenation of $b \Leftarrow a$ types. The diagrams are called **models** and are stored in model-ADT files analogous to `mm.mm`. With a diagram rich in paths, mm-ADT application code will tend towards a *look-and-feel* similar in form to

$$a = [b \Rightarrow c,\ d[x][y][z] \Rightarrow e \Rightarrow f] \Rightarrow \ldots \dashv z.$$

where $d[x][y][z]$ denotes some intermediate instructions that operate on d prior to translating d to e (i.e., an inline type path) and the connectives that reflect the core operators of the underlying stream ring are:

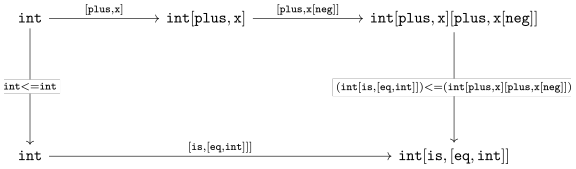
- `=>` : multiplicative monoid for serial composition
- `=[` : additive group for parallel alignment
- `=|` : non-commutative group for barrier aggregation

Stream Paths

Property Graph Model 3

```
pg_3:('import' -> (mm -> ()),
'type'   -> (
 graph  -> (graph<=edge{*}),
 vertex -> (vertex:('id'->int,'label'->str),
            vertex<=int-<('id'->_,'label'->'vertex')),
 edge   -> (edge:('outV'->vertex,'label'->str,'inV'->vertex),
            edge<=(vertex;vertex)-<('outV'->.0,'label'->'edge','inV'->.1))))
```

Higher Order Paths



Type Patterns

type	description	mmlang example

anonymous	A type with an unspecified domain.	`mmlang> 5 => [plus,2]`
		`==>7`
		`mmlang> 5 => [plus,[plus,2]]`
		`==>12`

monomial	A primitive type that is a single term and coefficient.	`mmlang> 5   => int`
		`==>5`
		`mmlang> 5   => int{10}`
		`language error: int is not an int{10}`

polynomial	A composite type containing a linearly combination of terms and their coefficients.	`mmlang> (+{2}3,+{3}4,+{4}5)`
		`==>(_[plus,3]{2},_[plus,4]{3},_[plus,5]{4})`
		`mmlang> 5-<(+{2}3,+{3}4,+{4}5)`
		`==>(8{2},9{3},10{4})`
		`mmlang> 5-<(+{2}3,+{3}4,+{4}5)>-[sum]`
		`==>83`

type	description	mmlang example

refinement	A subset of another type.	`mmlang> :[define,nat<=int[is>0]]`
		`==>_[define,nat<=int[is,bool<=int[gt,0]]]`
		`mmlang> 5 => nat`
		`==>nat:5`
		`mmlang> 0 => nat`
		`language error: 0 is not a nat`

recursive	A type with components of the same type.	`mmlang> :[model,mm][define,list<=[(_){?}|(_,list)]]`
		`==>_[define,list<=_[(_){?}|(_,list)]]`
		`mmlang> (1)  => list`
		`==>list:(1)`
		`mmlang> (1,(1))  => list`
		`==>list:(1,(1))`
		`mmlang> (1,(1,(1)))  => list`
		`==>list:(1,(1,(1)))`
		`mmlang> (1,(1,(1,(1)))) => list`
		`==>list:(1,(1,(1,(1))))`
		`mmlang> 1  => list`
		`language error: 1 is not a list`
		`mmlang> (1,1)  => list`
		`language error: (1{2}) is not a list`

dependent	A type with a definition variable to the incoming `obj`.	`mmlang> 5 => [is>int]`
		`mmlang> 5 => [plus,int]`
		`==>10`

model	A set of types and path equations.	`mmlang> :[model,social:(`
		`  'import' -> (mm -> ()),`
		`  'type'->`
		`  (person -> (person:('name'->str,'age'->nat)),`
		`  nat  -> (nat<=int[is>0])))]`
		`==>_`
		`mmlang> ('name'->'marko','age'->0)  => person`
		`language error: ('name'->'marko','age'->0) is not a person`
		`mmlang> ('name'->'marko','age'->29)  => person`
		`==>person:('name'->'marko','age'->nat:29)`

Refinement Types

Refinement types extend a language's base types with predicates that further *refine* (constrain) the base type values. A classic example is the set of natural numbers ($\mathbb{N}$) as a refinement of the set of integers ($\mathbb{Z}$), where $\mathbb{N} \subset \mathbb{Z}$. In set builder notation, specifying the set of integers and a predicate to limit the set to only those integers greater than 0 is denoted

$$\mathbb{N} = \{n \in \mathbb{Z} \mid n > 0\}.$$

In mm-ADT, the above is written `int[is>0]` which is the sugar form of `int{?}<=int[is,[gt,0]]`.

```
mmlang> :[model,mm][define,nat<=int[is>0]]
==>_[define,nat<=int[is,bool<=int[gt,0]]]
mmlang> 10 => nat
==>nat:10
mmlang> -1 => nat
language error: -1 is not a nat
mmlang> 10 => nat[plus,5]
==>nat:15
mmlang> 10 => nat[plus,5][plus,-15]
language error: 0 is not a nat
```

Dependent Types

```
mmlang> :[model,mm][define,vec:(lst,int)<=lst-<(_,=(_)>-[count]),
                                single<=vec:(lst,is<4).0[tail][head],
                                single<=vec:(lst,is>3).0[head]]
==>_[define,vec<=lst[split,(lst,int<=lst[combine,(_)][merge][count])],single<=vec:(lst,_[is,_[lt,4]])
[get,0,_][tail][head],single<=vec:(lst,_[is,_[gt,3]])[get,0,_][head]]
mmlang> (1;2;3)   => vec                                          ❶
==>vec:((1;2;3),1)
mmlang> (1;2;3)   => vec => single                                ❷
==>single:1
mmlang> (1;2;3;4) => vec                                          ❸
==>vec:((1;2;3;4),1)
mmlang> (1;2;3;4) => vec => single                                ❹
==>single:1
```

❶ A `;-lst` of 3 terms is morphed into a `vec` using the `vec<=lst` type.

❷ The `vec` is morphed into a `single` using the first `single<=vec` type.

❸ A `;-lst` of 4 terms is morphed into a `vec`.

❹ The `vec` is morphed into a `single` using the second `single<=vec` type.

Recursive Types

A recursive type's definition contains a reference to itself. Recursive type definitions require a *base case* to prevent an infinte recursion. Modern programming languages support generic collections, where a list can be defined to contain a particular type. For example, a `lst` containing only `int`s.

```
mmlang> :[model,mm][define,xlist<=lst[[is,[empty]]|
                                      [[is,[head][a,str]];
                                       [is,[tail][a,xlist]]]]]
==>_[define,xlist<=lst[lst{?}<=lst[is,bool<=lst[empty]]|lst{?}<=lst[lst{?}<=lst[is,bool<=lst[head]
[a,str]];lst{?}[is,bool{?}<=lst{?}[tail][a,xlist]]]]]
mmlang> ( ) => [a,xlist]
==>true
mmlang> ('a';'b';'c') => [a,xlist]
==>true
mmlang> ('a';'b';'c') => xlist
==>xlist:('a';'b';'c')
mmlang> (1;'a';'c') => xlist
language error: (1;'a';'c') is not a xlist
mmlang> ('a';'b';'c') => xlist[put,0,3]
language error: (3;'a';'b';'c') is not a xlist
```

```
mmlang> :[model,mm][define,ylist<=lst[[is,[empty]]|
```

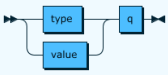
```
                                    [[is,[head][a,str]];
                                     [is,[tail][head][a,int]];
                                     [is,[tail][tail][a,ylist]]]]]
==>_[define,ylist<=lst[lst{?}<=lst[is,bool<=lst[empty]]|lst{?}<=lst[lst{?}<=lst[is,bool<=lst[head]
[a,str]];lst{?}[is,bool{?}<=lst{?}[tail][head][a,int]];lst{?}[is,bool{?}<=lst{?}[tail][tail][a,ylist]]]]]
mmlang> ( ) => [a,ylist]
==>true
mmlang> ('a';1;'b';2) => [a,ylist]
==>true
mmlang> ('a';1;'b';2) => ylist
==>ylist:('a';1;'b';2)
mmlang> (1;'a';'c') => ylist
language error: (1;'a';'c') is not a ylist
mmlang> ('a';1;'b';2) => ylist[put,0,3]
language error: (3;'a';1;'b';2) is not a ylist
```

Reference

mmlang Grammar

```
obj   ::= (type | value)q
value ::= vbool | vint | vreal | vstr
vbool ::= 'true' | 'false'
vint  ::= [1-9][0-9]*
vreal ::= [0-9]+'.'[0-9]*
vstr  ::= "'" [a-zA-Z]* "'"
type  ::= ctype | dtype
ctype ::= 'bool' | 'int'  | 'real' | 'str' | poly | '_'
poly  ::= lst | rec | inst
q     ::= '{' int (',' int)? '}'
dtype ::= ctype q? ('<=' ctype q?)? inst*
sep   ::= ';' | ',' | '|'
lst   ::= '(' obj (sep obj)* ')' q?
rec   ::= '(' obj '->' obj (sep obj '->' obj)* ')' q?
inst  ::= '[' op(','obj)* ']' q?
op    ::= 'a' | 'add' | 'and' | 'as' | 'combine' | 'count' | 'eq' | 'error' |
          'explain' | 'fold' | 'from' | 'get' | 'given' | 'groupCount' | 'gt' |
          'gte' | 'head' | 'id' | 'is' | 'last' | 'lt' | 'lte' | 'map' | 'merge' |
          'mult' | 'neg' | 'noop' | 'one' | 'or' | 'path' | 'plus' | 'pow' | 'put' |
          'q' | 'repeat' |'split' | 'start' | 'tail' | 'to' | 'trace' | 'type' | 'zero'
```

obj:

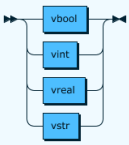



```
obj     ::= ( type | value ) q
```

referenced by:

- inst
- lst
- rec

value:

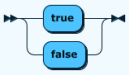



```
value   ::= vbool
          | vint
          | vreal
          | vstr
```

referenced by:

- obj

vbool:




```
vbool   ::= 'true'
          | 'false'
```

referenced by:

- value

vint:

Instructions

The mm-ADT VM instruction set architecture is presented below, where the instructions are ordered by their classification and within each classification, they are ordered alphabetically.

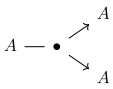
Branch Instructions

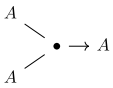
Branch instructions enable the splitting, composing, and merging of streams. This subset of `inst` is a particular type of additive category called a traced monoidal category, where `[repeat]` provides feedback and each instruction's `poly` arguments are biproducts maintaining both injective and projective morphisms. There exists a well-established graphical language for such monoidal categories that has been adopted in `mmlang` sugar syntax (or as best as can be reasonably captured using ASCII characters).

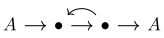
name	string diagram	sugar	mmlang example

initial	$\mathbf{0} \longrightarrow A$	`A`	`mmlang> int`
			`==>int`
			`mmlang> 6`

name	string diagram	sugar	mmlang example
			`==>6`

split		`-<(A,A)`	`mmlang> int-<(_,_)`
			`==>(int{2}<=int[id]{2})<=int[split,(int{2}<=int[id]{2})]`
			`mmlang> 6-<(int,int)`
			`==>(6{2})`

merge		`(A,A)>-`	`mmlang> (int,int)>-`
			`==>int{2}<=(int{2}<=int[id]{2})[merge]`
			`mmlang> (6,6)>-`
			`==>6{2}`

repeat		`(A)^(A)`	`mmlang> int(+3)^(5)`
			`==>int[repeat,int[plus,3],5]`
			`mmlang> 6(+3)^(5)`
			`==>21`

terminal		`A{0}`	`mmlang> int{0}`
			`mmlang>`
			`mmlang> 6{0}`
			`mmlang>`

inst	description	example

`[branch]`
`[ ]`
	Juxtapose start across `poly` terms and then aggregate the terms of the resultant `poly` as dictated by the `poly`s underlying magma (obj$_{LR}$-module).
	$$x(a_0, a_1, \ldots, a_n) = \bigodot_{i=0}^{i \le n} x a_i$$
		`mm> 6[branch,(+1,'a',*int)]`
		`==>7`
		`==>'a'`
		`==>36`
		`mm> 6[+1,'a',*int]`
		`==>7`
		`==>'a'`
		`==>36`

`[combine]`
`=`
	Pairwise juxtapose the terms of two `poly`s (Hadamard product and obj[X]).
	$$(a_0, \ldots, a_n) \circ (b_0, \ldots, b_n) = (a_0 b_0, \ldots, a_n b_n)$$
		`mm> (1;2;3)[combine,(+10;+20;+30)]`
		`==>(11;22;33)`
		`mm> (1;2;3)=(+10;+20;+30)`
		`==>(11;22;33)`

`[lift]` `<< >>`	Detach from the `obj` graph and operate stateless.	

`[merge]`
`>-`
	Aggregate the terms of the start `poly`, where aggregation semantics is determined by the underlying magma of the `poly` (obj$_R$-module).
	$$(a_0, a_1, \ldots, a_n) = \bigodot_{i=0}^{i \le n} a_i$$
		`mm> (1,2,3)[merge]`
		`==>1`
		`==>2`
		`==>3`
		`mm> (1,2,3)>-`
		`==>1`
		`==>2`
		`==>3`
		`mm> (1;2;3)>-`
		`==>3`
		`mm> (1|2|3)>-`
		`==>1`

`[repeat]`
`( )^( )`
	Apply the first `obj` (typically a type) to the incoming `obj` until the second `obj` is not `{0}`. For ease of use, if the second `obj` is an `int` value, then this is interpreted as number of times to iterate.
	$$x(a^n) = x \prod_{i=0}^{i \le n} a$$
		`mm> [1,2,3][repeat,+2,3]`
		`==>7`
		`==>8`
		`==>9`
		`mm> (1,2,3)>-(+2)^(3)`
		`==>7`
		`==>8`
		`==>9`
		`mm> (1,2,3)>-(*2)^(is<10)`
		`==>16{2}`
		`==>12`

`[split]`
`-<`
	Juxtapose start across `poly` terms as a scalar to a vector. `[branch]` is equivalent to `[split][merge]` (obj$_{;}$-module).
	$$x(a_0, a_1, \ldots, a_n) = (x a_0, x a_1, \ldots, x a_n)$$
		`mm> 1[split,(+1,+2,+3)]`
		`==>(2,3,4)`
		`mm> 1-<(+2,+3,+4)`
		`==>(3,4,5)`
		`mm> 1-<(+2;+3;+4)`
		`==>(3;6;10)`
		`mm> 1-<(+2|+3|+4)`
		`==>(3)`

`[swap]`
`/ /`
	Replace the `[swap]` type's last `inst` argument with the start and the start with the `inst` argument (swap).
	$$xa = ax$$
		`mm> '-adt'[plus,'mm']`
		`==>'-adtmm'`
		`mm> '-adt'[swap,[plus,'mm']]`
		`==>'mm-adt'`
		`mm> '-adt'+'mm'`
		`==>'-adtmm'`
		`mm> '-adt'/+'mm'/`
		`==>'mm-adt'`

Filter Instructions

Map Instructions

Reduce Instructions

Trace Instructions